

Neural Network and Deep Learning (10 hrs)

5.1. Perceptron, Multi-Layer Perceptron (MLP) and Backpropagation

5.2. CNN (Convolutional Neural Networks)- Zero-shot, Few shot learning, Graph CNN

5.3. RNN (Recurrent Neural Networks)

5.4. Attention Mechanisms

5.5. Transformers

5.6. Graph Attention Networks

5.7. Transfer Learning

Introduction to Artificial Intelligence (AI)

Artificial Intelligence (AI) is the branch of computer science concerned with building systems that can perform tasks that normally require human intelligence for **learning, reasoning, problem-solving, perception, understanding language, and even decision-making.**

Types of AI

1. Narrow AI (Weak AI)

- Performs a specific task (e.g., facial recognition, spam filtering)
- Example: Google Translate

2. General AI (Strong AI)

- Mimics human cognitive abilities across a broad range of tasks (still theoretical)

3. Superintelligent AI

- Surpasses human intelligence in all respects (future concept)

Core Idea

AI aims to create machines that can:

- **Think logically**
- **Learn from experience**
- **Adapt to new inputs**
- **Act autonomously**

Applications of AI

- **Healthcare:** Disease prediction, drug discovery
- **Finance:** Fraud detection, algorithmic trading
- **Transportation:** Self-driving cars
- **Customer Service:** Chatbots, recommendation systems
- **Education:** Intelligent tutoring systems

Subfields of AI

- **Machine Learning (ML)** Algorithms that improve automatically through experience
- **Natural Language Processing (NLP)** Understanding and generating human language
- **Computer Vision** Interpreting and processing visual information
- **Robotics** AI embedded in physical agents to interact with the environment
- **Expert Systems** Rule-based systems that mimic decision-making of experts
- **Recommender and optimization system**

Components of an Expert System

An **Expert System** is a computer program that simulates the **decision-making ability** of a human expert.

It uses **knowledge** and **inference** to solve complex problems that usually require human expertise.

Here are the **main components** of an expert system:

Component	Role
Knowledge Base	Stores domain-specific rules and facts
Inference Engine	Applies rules to derive conclusions
User Interface	Communication bridge between user and system
Explanation Facility	Explains the logic behind decisions
Knowledge Acquisition	Updates or adds knowledge from human experts
Working Memory	Temporarily stores current inputs and intermediate results

Knowledge-Based Expert System (KBES)

A Knowledge-Based Expert System is a type of Artificial Intelligence system that simulates human expertise by using a knowledge base and inference rules/ agorithum to solve complex problems in a specific domain.

Definition

A Knowledge-Based Expert System (KBES) is a computer program that:

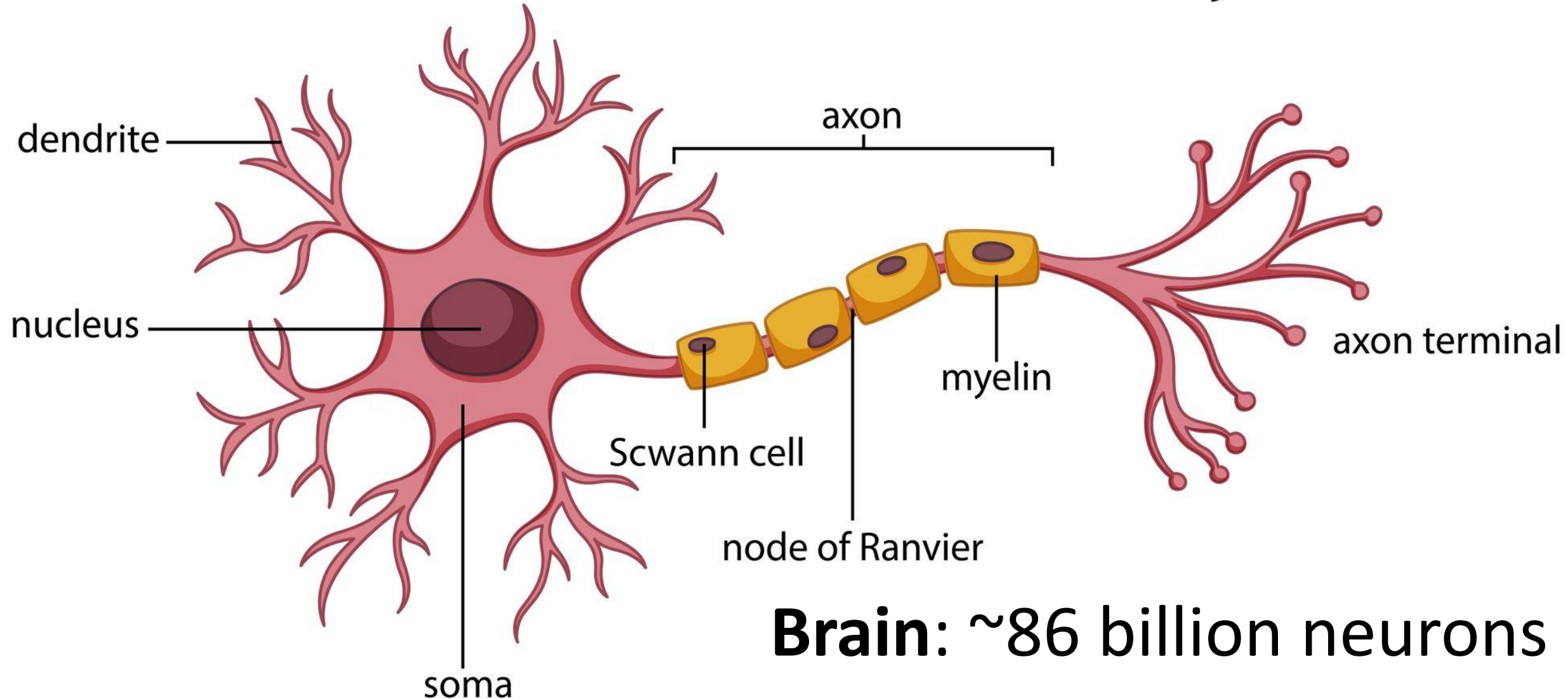
- Uses stored knowledge about a particular domain
- Applies reasoning techniques (logic and inference)
- Provides solutions, explanations, or recommendations
- Emulates decision-making abilities of a human expert

Key Features

- 1.Rule-based reasoning (e.g., IF-THEN rules)
- 2.Knowledge representation (facts, rules, heuristics)
- 3.Domain-specific expertise
- 4.Interactive interface for querying and explaining solutions

Component	Description
Knowledge Base	Stores domain knowledge (facts + rules)
Inference Engine	Applies logic to infer conclusions from knowledge base
User Interface	Allows users to interact with the system (enter problems, receive answers)
Explanation System	Justifies reasoning and outputs
Knowledge Acquisition	Helps add/update expert knowledge

Neuron Anatomy



Brain: ~86 billion neurons

What is an Artificial Neural Network (ANN)?

An **Artificial Neural Network (ANN)** is a **computational model** inspired by the way biological brains process information. It is a key technology in **machine learning** and **artificial intelligence (AI)**, particularly for tasks like pattern recognition, image processing, speech recognition, and predictive forecasting and estimation modeling.

Definition

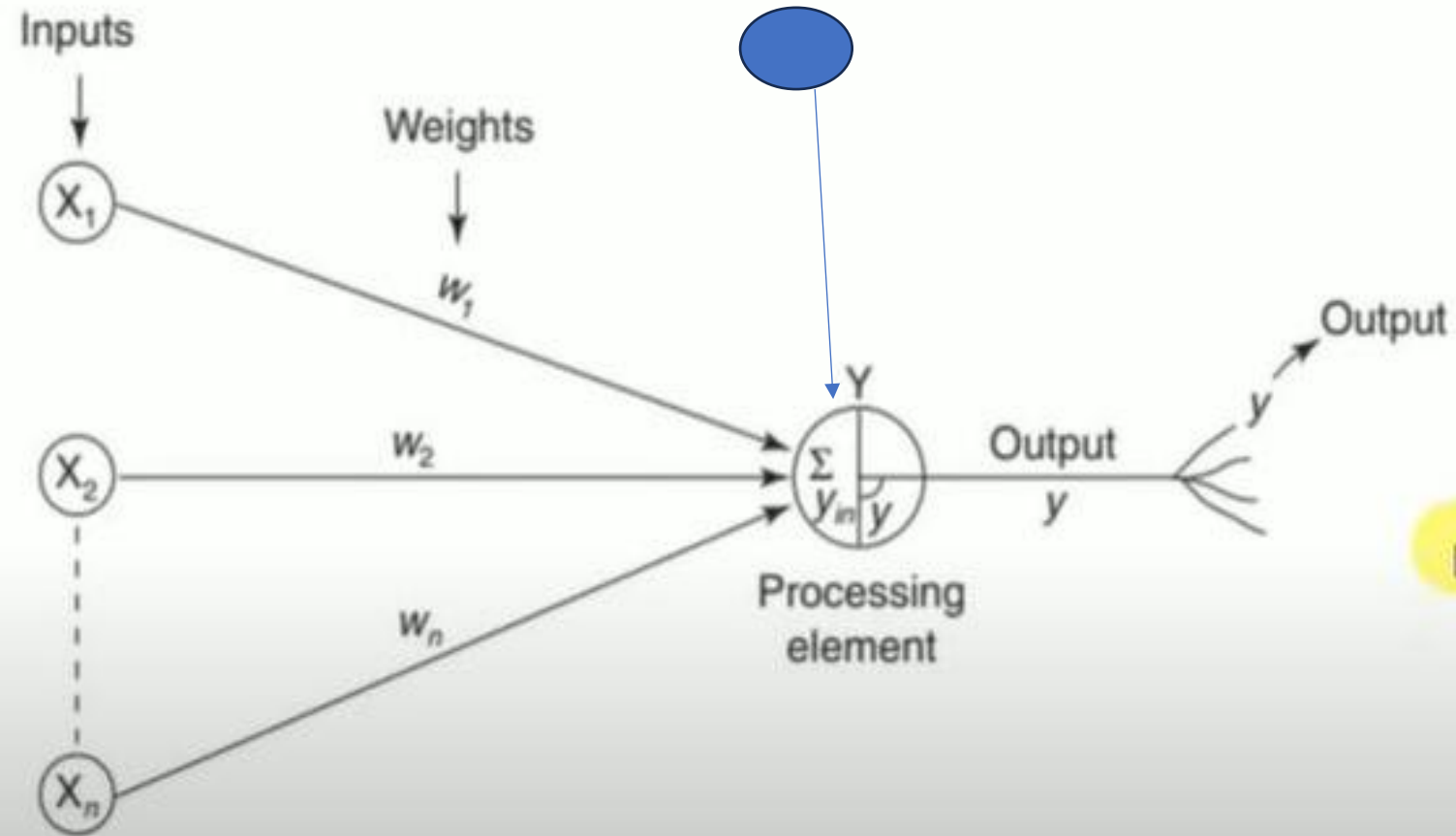
An **Artificial Neural Network** is a network of **interconnected nodes (called neurons)** that work together to recognize patterns and solve complex problems by **learning from data**.

How ANN Works

- 1. Input data is fed into the **input layer**.
- 2. It flows through one or more **hidden layers** where calculations happen.
- 3. Each neuron applies **weights**, sums the inputs, and passes the result through an **activation function**.
- 4. The **output layer** gives the **prediction or classification**.
- 5. During training, the model uses **backpropagation** and **gradient descent** to adjust the weights and reduce error.

Component	Description
Neurons	Basic units (like brain cells) that receive input, process it, and pass output
Input Layer	Receives raw data (e.g., images, numbers)
Hidden Layers	Layers between input and output that perform computations and learning
Output Layer	Produces final result or prediction (e.g., classification label)
Weights	Parameters that determine the importance of inputs
Activation Function	Decides whether a neuron should be "fired" or not (e.g., sigmoid, ReLU)

Artificial Neural Networkⁱ

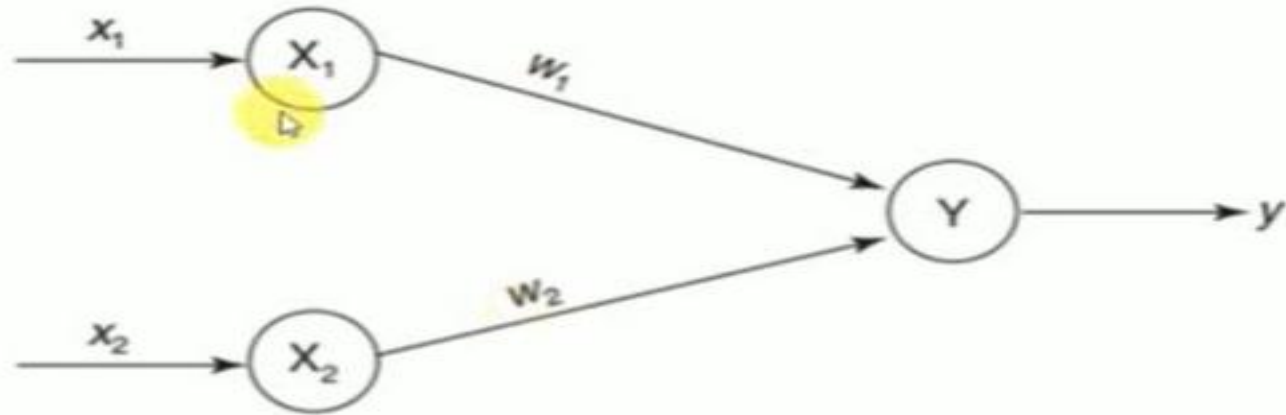


Introduction

Artificial Neural Network: Introduction

- An artificial neural network (ANN) may be defined as an information-processing model that is inspired by the way biological nervous systems, such as the brain, process information.
- This model tries to replicate only the most basic functions of the brain.
- An ANN is composed of a large number of highly interconnected processing units (neurons) working in unison to solve specific problems.
- Like human being, Artificial neural networks learn by example.
- An ANN is configured for a specific application, such as spam classification, Face Recognition, pattern recognition through a learning process.
- Each neuron is connected with the other by a connection link.
- Each connection link is associated with weights which contain information about the input signal.
- This information is used by the neuron network to solve a particular problem.
- ANNs' collective behavior is characterized by their ability to learn, recall and generalize training patterns or data similar to that of a human brain.
- They have the capability to model networks of original neurons as found in the brain.
- Thus, the ANN processing elements are called neurons or artificial neurons.

- Each neuron has an internal state of its own.
- This internal state is called the activation level of neuron
- The activation signal of a neuron is transmitted to other neurons.
- Remember, a neuron can send only one signal at a time, which can be transmitted to several other neurons.



- Input neurons X_1 and X_2 are connected to the output neuron Y , over a weighted interconnection links (w_1 and w_2).
- For the above simple neuron net architecture, the net input has to be calculated in the following way:

$$y_{in} = x_1 w_1 + x_2 w_2$$

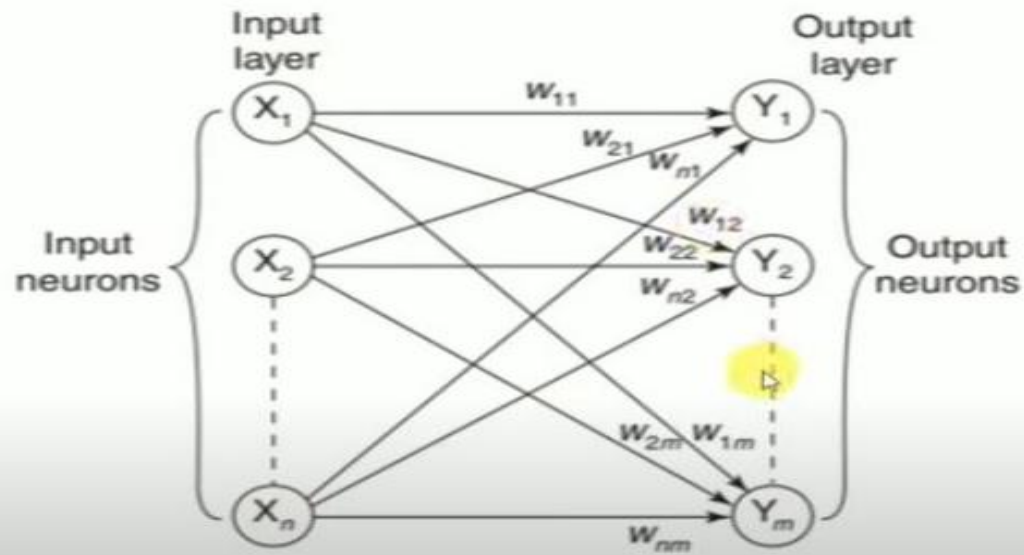
1. Connections

- An ANN consists of a set of highly interconnected processing elements (neurons) such that each processing element output is found to be connected through weights to the other processing elements or to itself; delay lead and lag-free connections are allowed.
- Hence, the arrangements of these processing elements and the geometry of their interconnections are essential for an ANN.

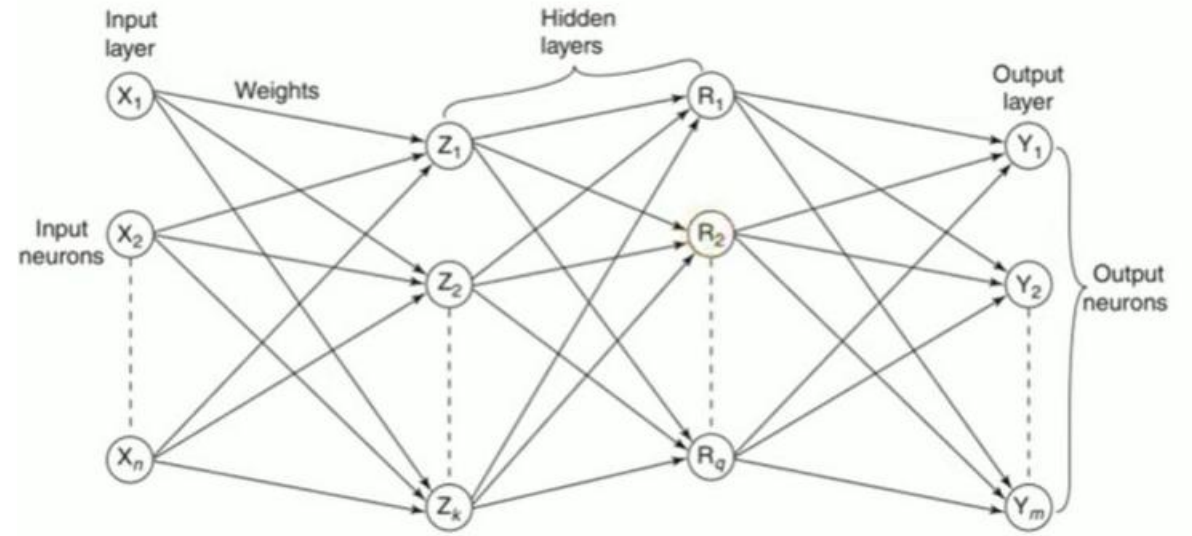
There exist five basic types of neuron connection architectures. They are:

1. single-layer feed-forward network
2. multilayer feed-forward network
3. single node with its own feedback
4. single-layer recurrent network
5. multilayer recurrent network.

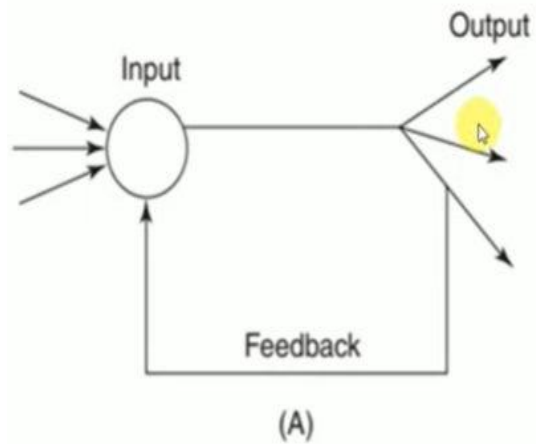
1. Connections



Single-layer feed-forward network.

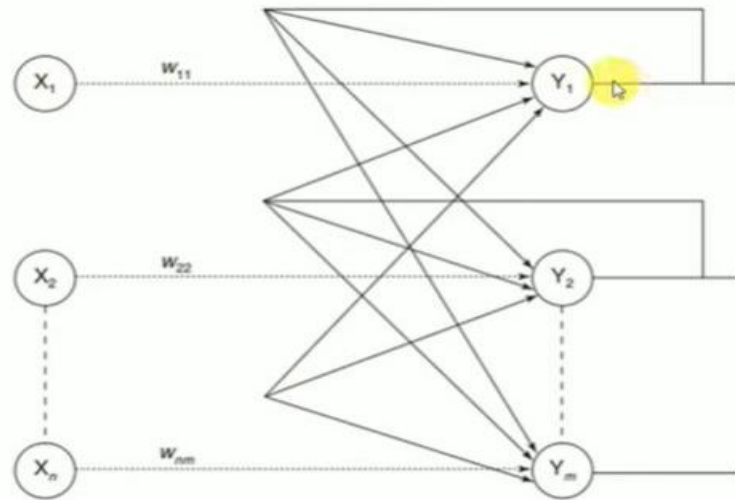


Multilayer feed-forward network.

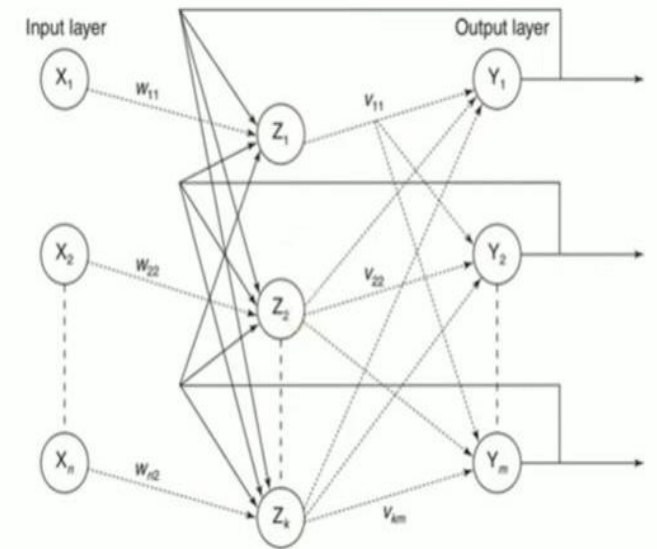


(A)

Single node with own feedback.



Single-layer recurrent network.



Multilayer recurrent network.

2. Learning

- The main property of an ANN is its capability to learn.
- Learning or training is a process by means of which a neural network adapts itself to a stimulus by making proper parameter adjustments, resulting in the production of desired response.

Gradient descent/Back propagator

- Broadly, there are two kinds of learning in ANNs:
 1. **Parameter learning:** It updates the connecting weights in a neural net.
 2. **Structure learning:** It focuses on the change in network structure (which includes the number of processing elements as well as their connection types).

3. Activation Functions

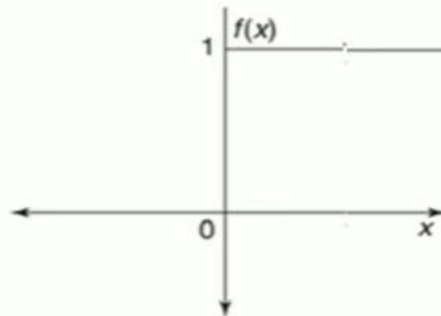
2. Binary step function: This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

where θ represents the threshold value.

This function is most widely used in single-layer nets

to convert the net input to an output that is a binary (1 or 0).



3. Activation Functions

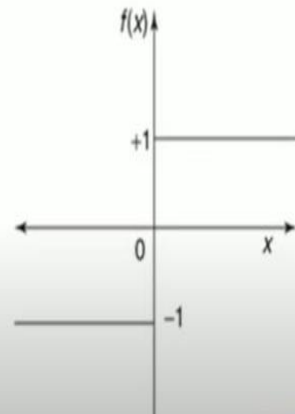
3. Bipolar step function: This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

where θ represents the threshold value.

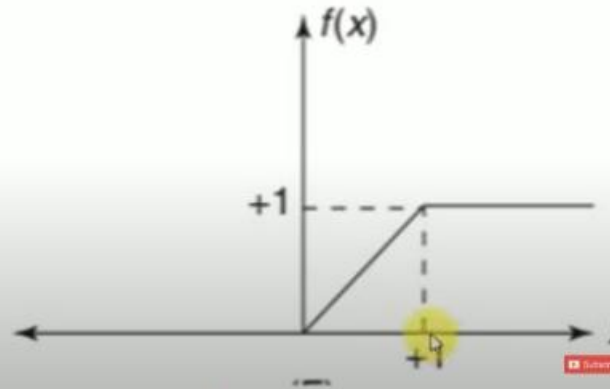
This function is also used in single-layer nets to convert

the net input to an output that is bipolar (+1 or -1).



4. Ramp function: The ramp function is defined as

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases}$$



5. Sigmoidal functions: The sigmoidal functions are widely used in back-propagation nets because of the relationship between the value of the functions at a point and the value of the derivative at that point which reduces the computational burden during training.

Sigmoidal functions are of two types:

- Binary sigmoid function
- Bipolar sigmoid function

5.1 Binary sigmoid function: It is also termed as logistic sigmoid function or unipolar sigmoid function. It can be defined as

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

where λ is the steepness parameter. The derivative of this function is

$$f'(x) = \lambda f(x)[1 - f(x)]$$

Here the range of the sigmoid function is from 0 to 1.

5.2 Bipolar sigmoid function: is defined as

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

where λ is the steepness parameter and

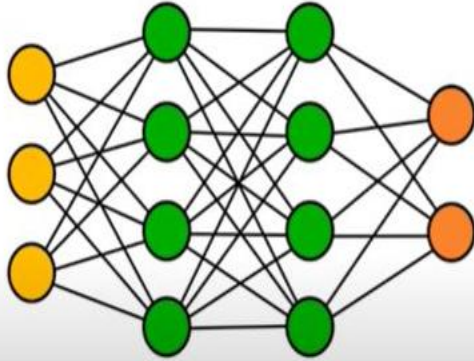
the sigmoid function range is between -1 and $+1$.

The derivative of this function can be

$$f'(x) = \frac{\lambda}{2} [1 + f(x)][1 - f(x)]$$

ANN – the basics

Status	PSA
Cancer	3.8
Cancer	3.4
Cancer	2.9
Cancer	2.8
Cancer	2.7
Cancer	2.1
Cancer	1.6
Healthy	2.5
Healthy	2.0
Healthy	1.7
Healthy	1.4
Healthy	1.2
Healthy	0.9
Healthy	0.8



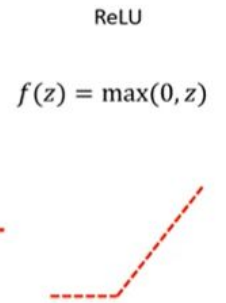
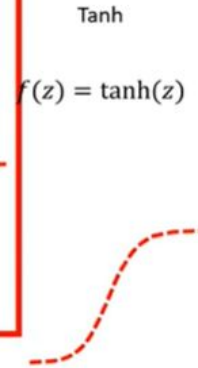
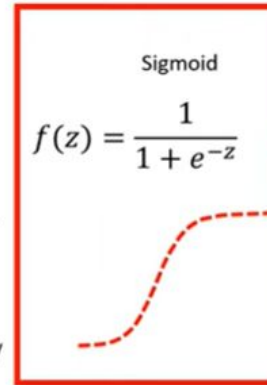
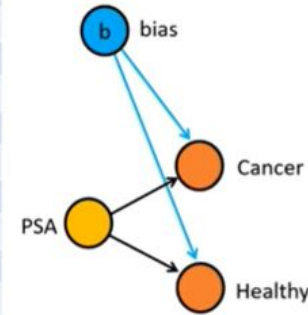
$$z = -5.745 + 2.747 \cdot 2.0 = -0.251$$

$$f(z) = \frac{1}{1 + e^{-(0.251)}} = 0.562$$

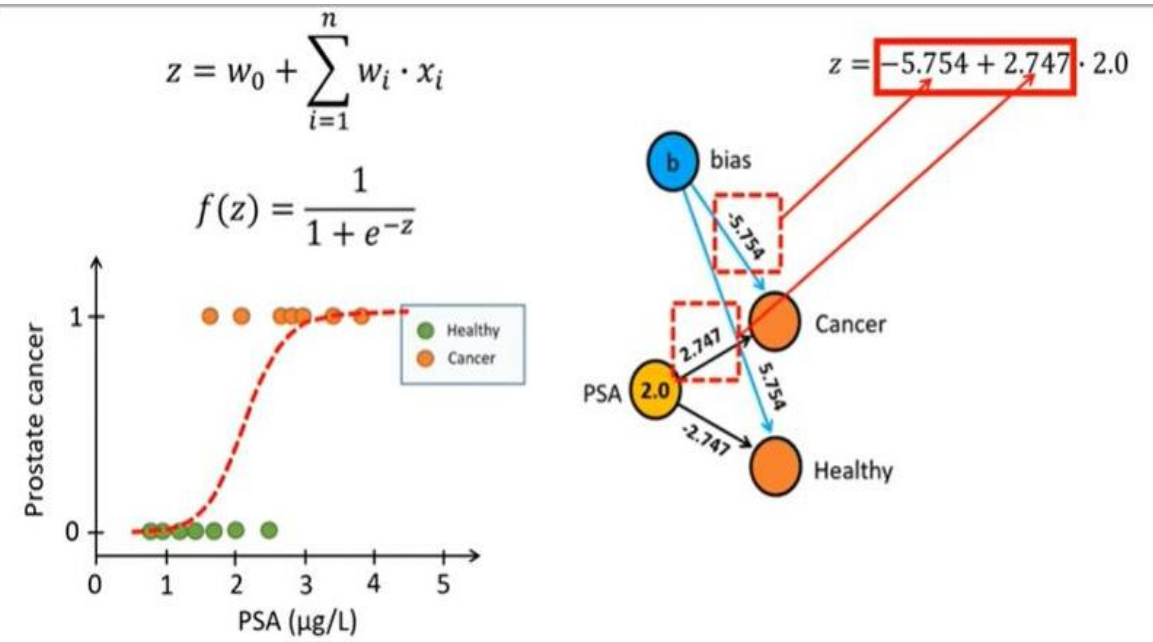
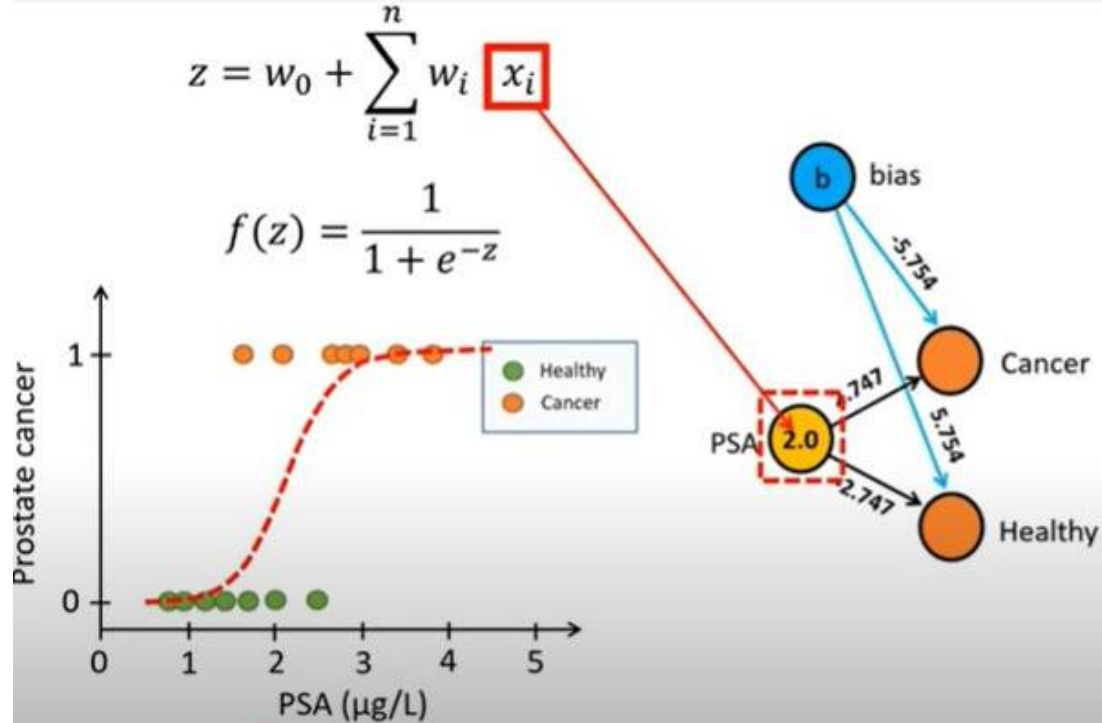
TileStats

Classification

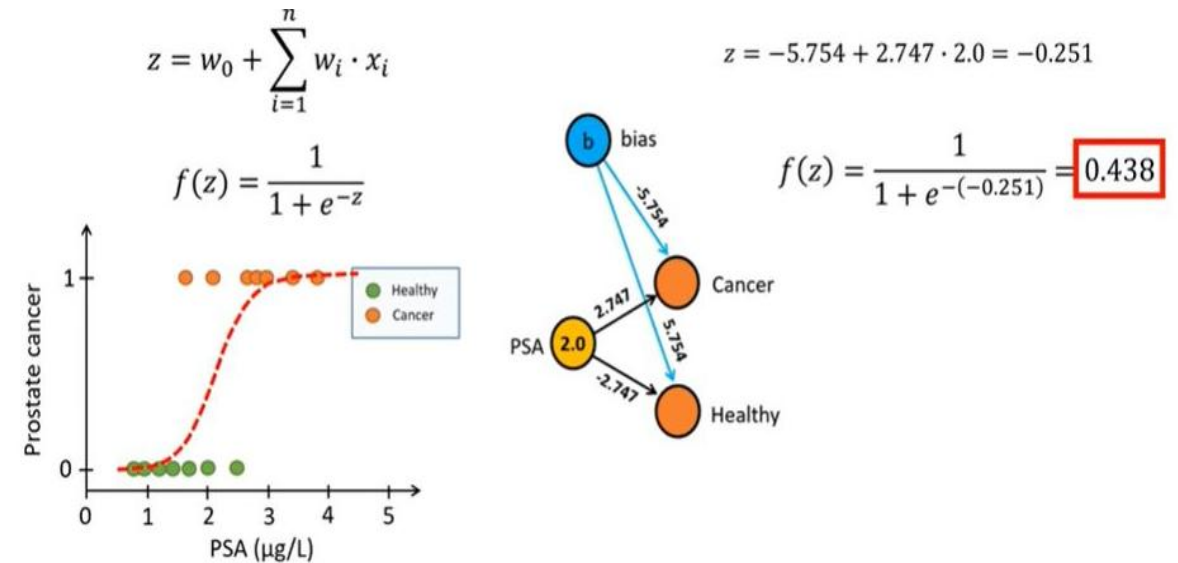
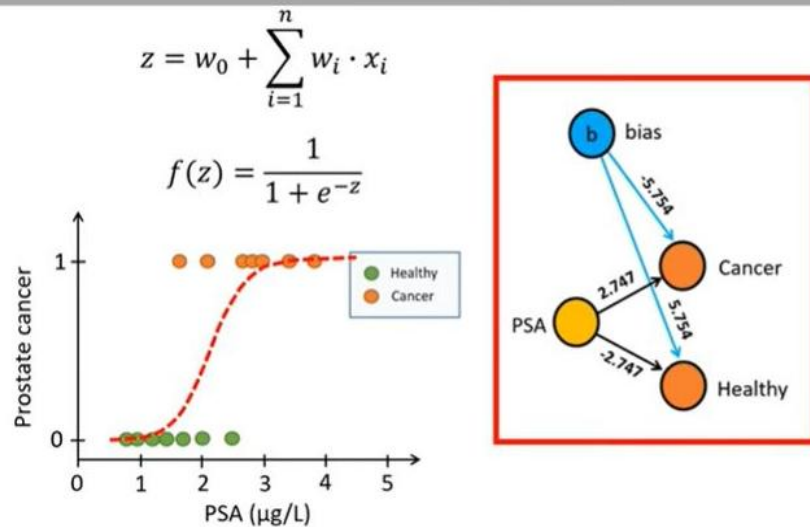
Status	PSA
Cancer	3.8
Cancer	3.4
Cancer	2.9
Cancer	2.8
Cancer	2.7
Cancer	2.1
Cancer	1.6
Healthy	2.5
Healthy	2.0
Healthy	1.7
Healthy	1.4
Healthy	1.2
Healthy	0.9
Healthy	0.8



We will here use the sigmoid function, which is the same function that is used in logistic regression. This will later allow us to compare the neural network with logistic regression.



Classification



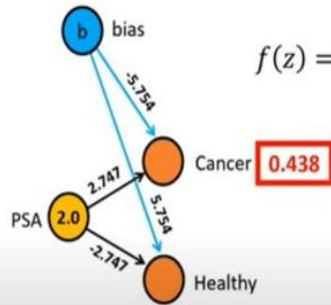
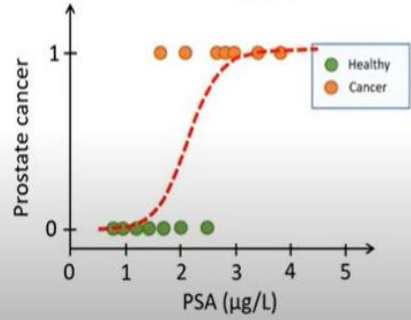
and calculate. This value is the value for the output node cancer, which can, in this case, be seen as the probability that the person has prostate cancer.

If we train this simple network, we will obtain the following weights.

Classification

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

$$f(z) = \frac{1}{1 + e^{-z}}$$



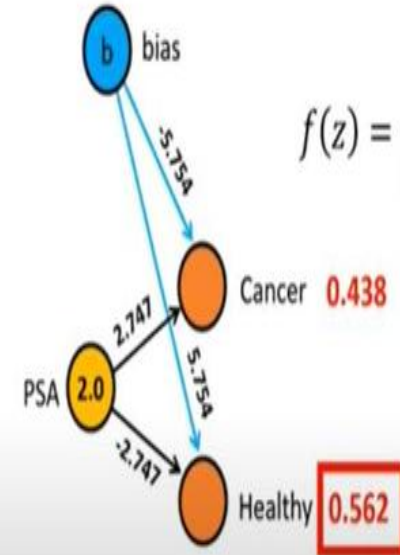
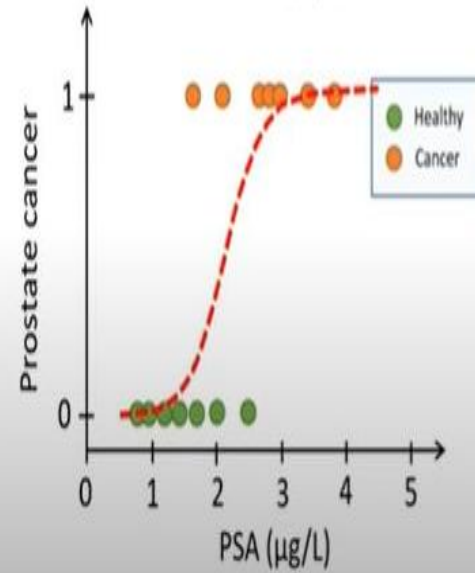
$$z = -5.754 + 2.747 \cdot 2.0 = -0.251$$

$$f(z) = \frac{1}{1 + e^{-(-0.251)}} = 0.438$$

Classification

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

$$f(z) = \frac{1}{1 + e^{-z}}$$



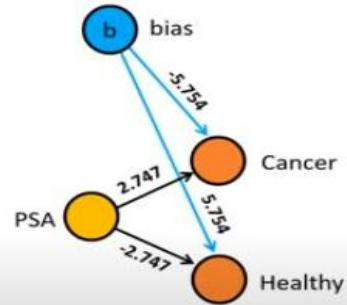
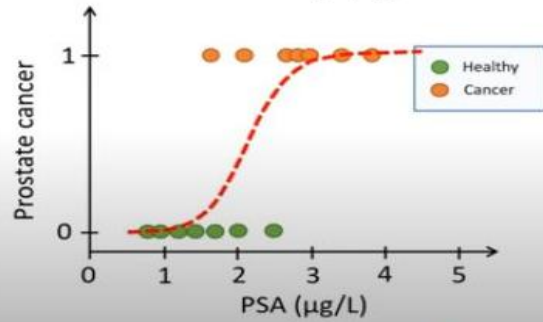
$$z = 5.754 + (-2.747) \cdot 2.0 = 0.251$$

$$f(z) = \frac{1}{1 + e^{-(0.251)}} = 0.562$$

Evaluation

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

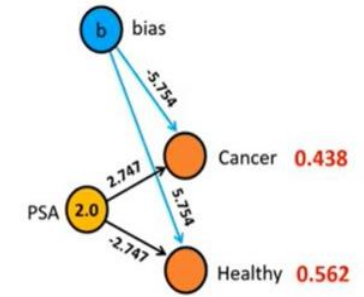
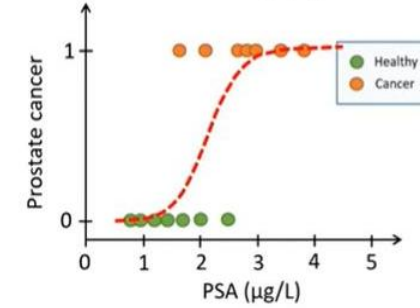
$$f(z) = \frac{1}{1 + e^{-z}}$$



Status	PSA	Cancer	Prediction
Cancer	3.8	0.991	Cancer
Cancer	3.4	0.973	Cancer
Cancer	2.9	0.902	Cancer
Cancer	2.8	0.875	Cancer
Cancer	2.7	0.842	Cancer
Cancer	2.1	0.506	Cancer
Cancer	1.6	0.206	Healthy
Healthy	2.5	0.755	Cancer
Healthy	2.0	0.438	Healthy
Healthy	1.7	0.254	Healthy
Healthy	1.4	0.130	Healthy
Healthy	1.2	0.079	Healthy
Healthy	0.9	0.036	Healthy
Healthy	0.8	0.028	Healthy

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

$$f(z) = \frac{1}{1 + e^{-z}}$$



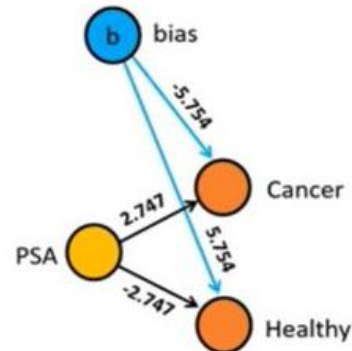
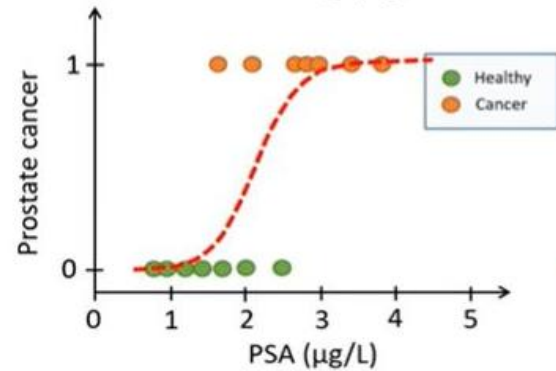
Cutoff = 0.5

A common cutoff value to use in binary classification is 0.5 because it is the value between zero and one.

Evaluation

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

$$f(z) = \frac{1}{1 + e^{-z}}$$



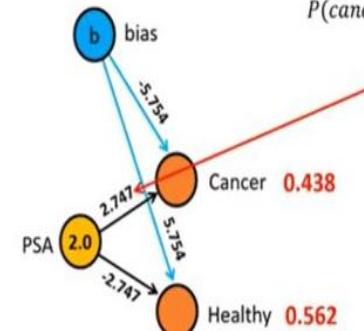
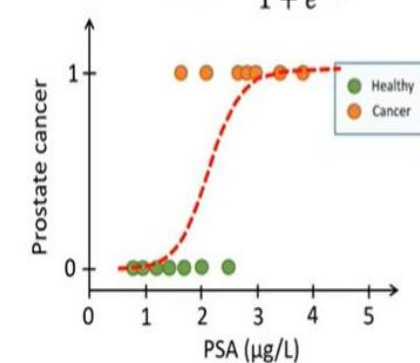
$$\text{Accuracy} = \frac{12}{14} = 86\%$$

which gives an accuracy of about 86%.

Logistic regression

$$z = w_0 + \sum_{i=1}^n w_i \cdot x_i$$

$$f(z) = \frac{1}{1 + e^{-z}}$$



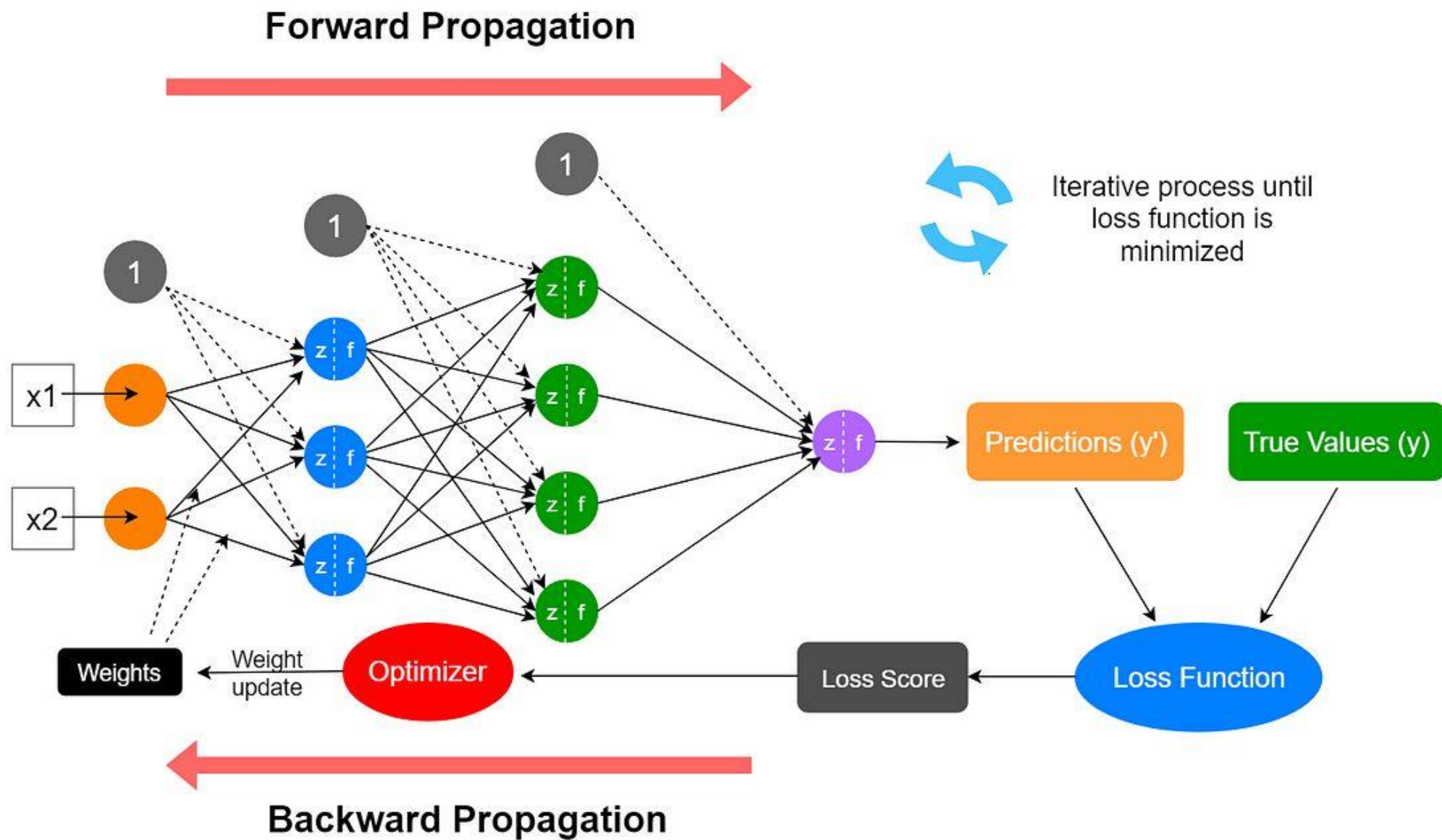
Logistic regression

$$P(\text{cancer}) = \frac{1}{1 + e^{-(b_0 + b_1 \cdot \text{PSA})}}$$

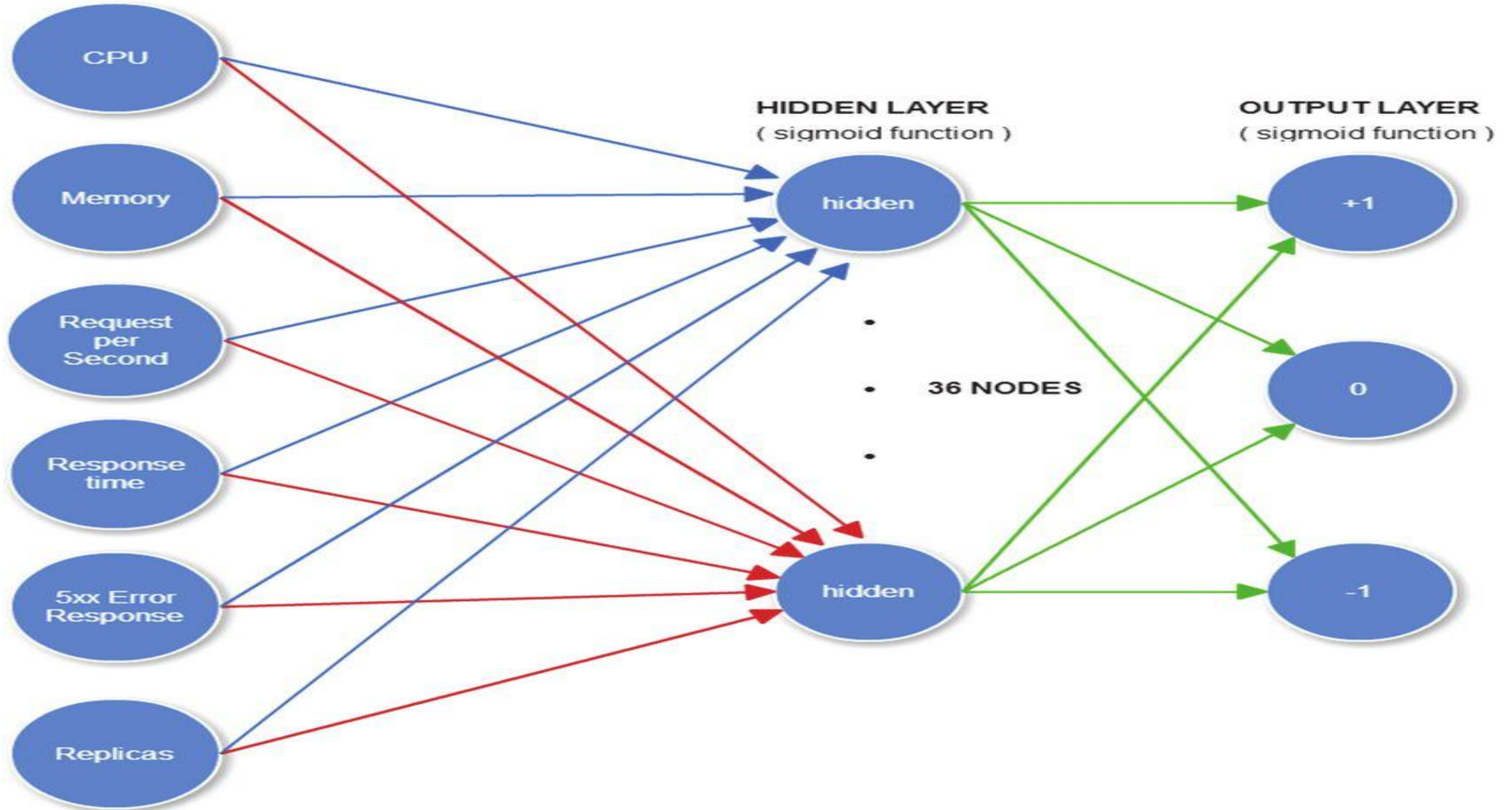
$b_0 = -5.754$
 $b_1 = 2.747$

whereas b_1 corresponds to the weight associated with the output node for prostate cancer.

Activation Function	Typical Use Case	Output Range	Suitable Data/Task Type
Sigmoid	Binary classification	$(0, 1)$	Binary targets, probabilities
Tanh	Hidden layers	$(-1, 1)$	Centered inputs, hidden layers
ReLU	Hidden layers	$[0, \infty)$	Non-negative, sparse, deep nets
Leaky ReLU/PReLU	Hidden layers	$(-\infty, \infty)$	Like ReLU, but avoid dead neurons
Softmax	Multi-class classification	$(0,1)$ sum=1	Multi-class categorical outputs
Linear	Regression	$(-\infty, \infty)$	Continuous output prediction



INPUT
(linear function)

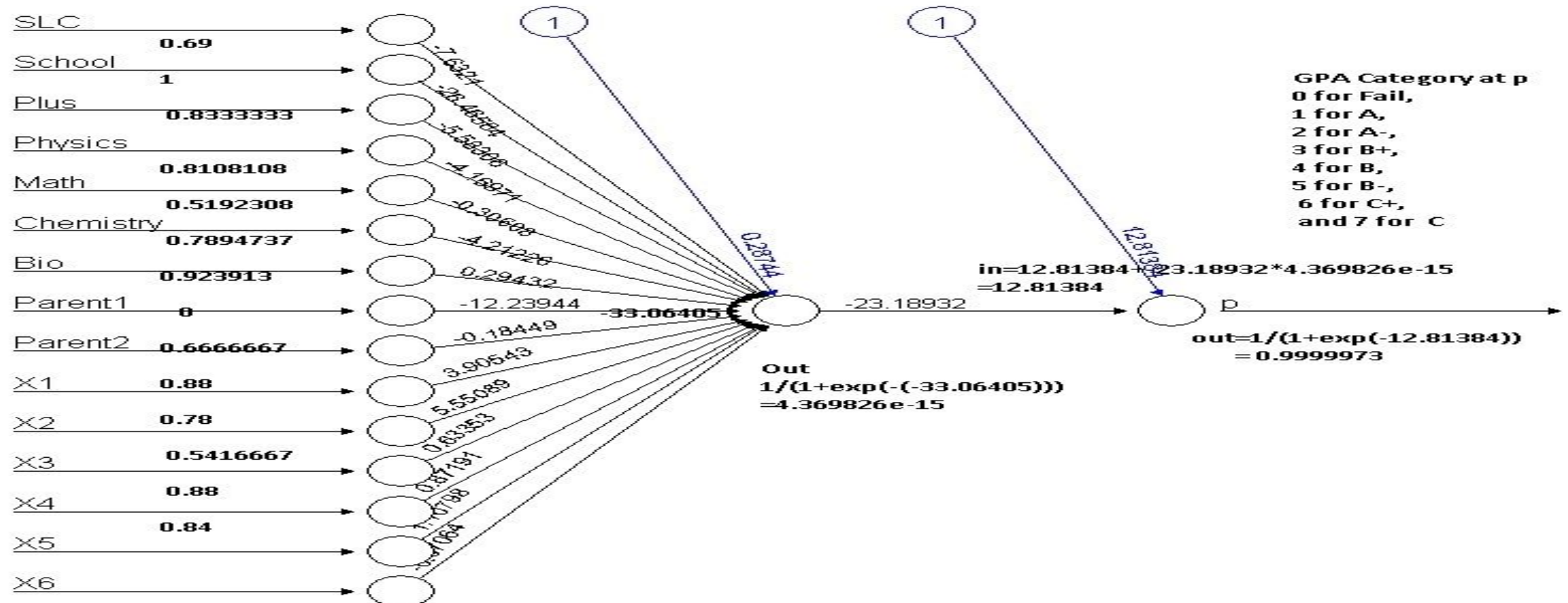


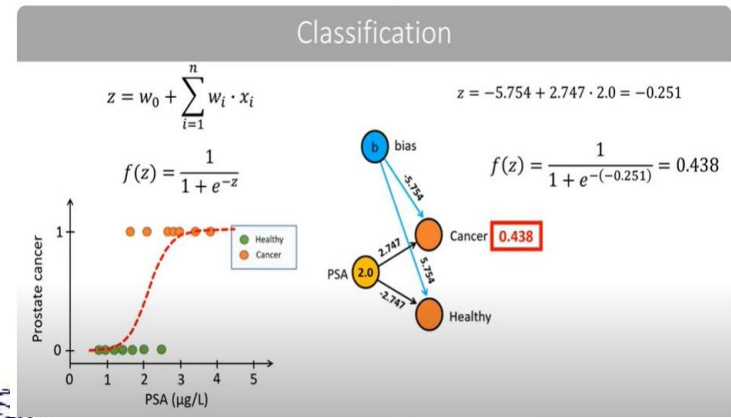
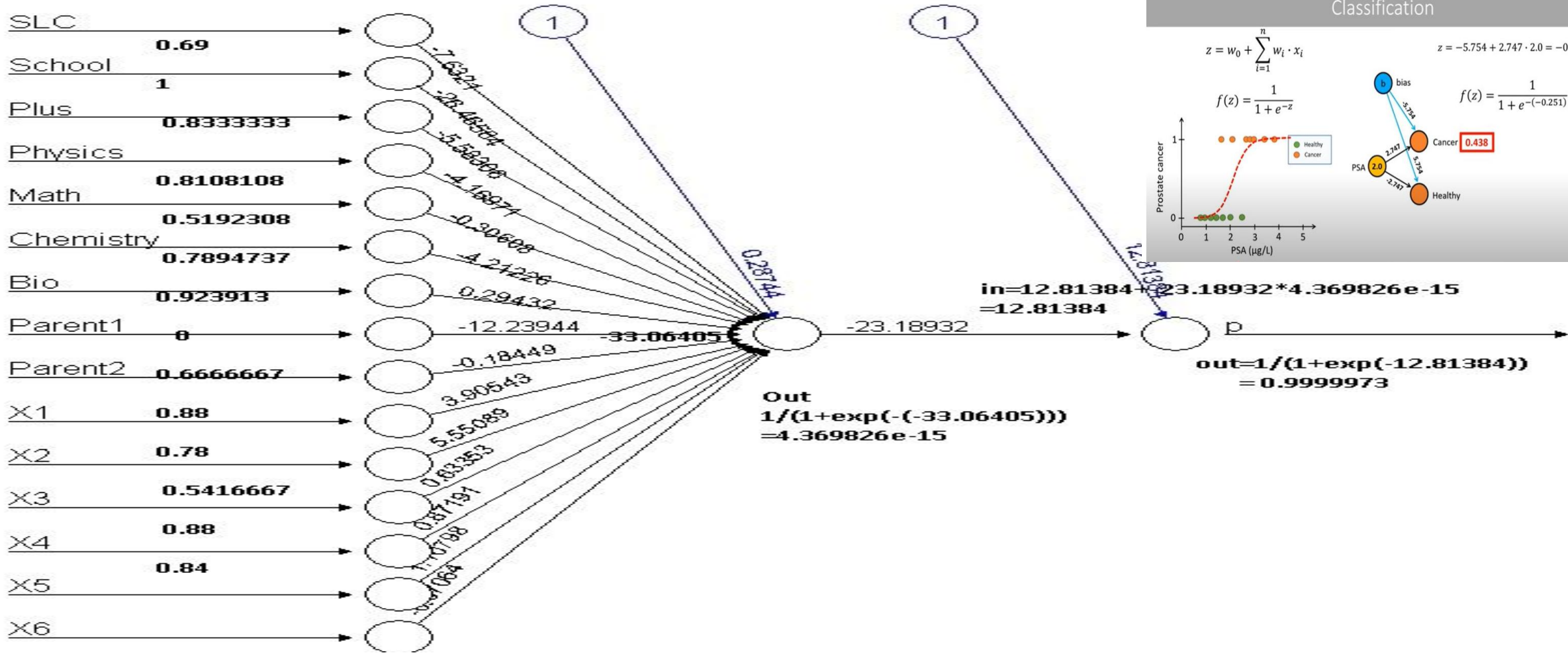
How Neural Network Model Works

Machine learning is an application of artificial intelligence (AI) that provides the ability to **automatically learn** and improve prediction **from past experiences without being explicitly programmed**.

Artificial neural networks, usually simply called neural networks, are computing systems inspired by the biological neural networks that **constitute animal brains**. An ANN is based on a collection of connected units or nodes called artificial neurons, which loosely model the neurons in a biological brain(Wikipedia,2020)

activation functions are the sigmoid and hyperbolic tangent





in14=0.28744s)+(1.10798*0.84)+(-0.07064*0.84)

=-23.166

Test of Deep Learning with Single Hyperparameter

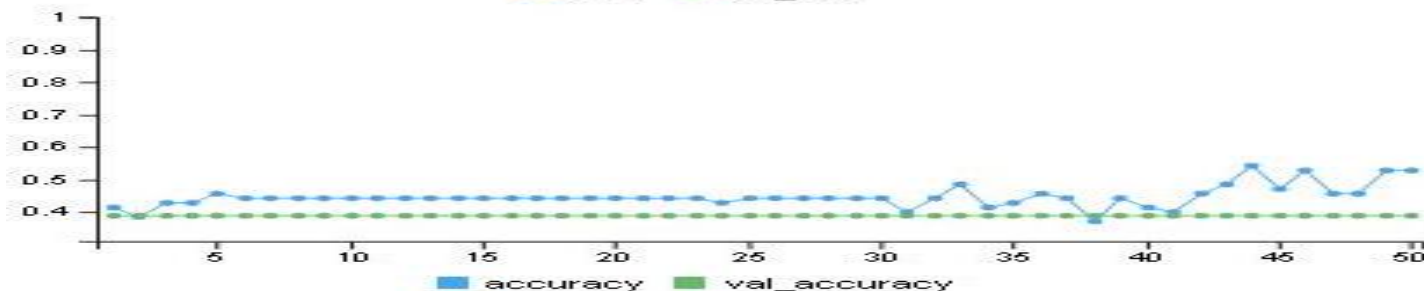
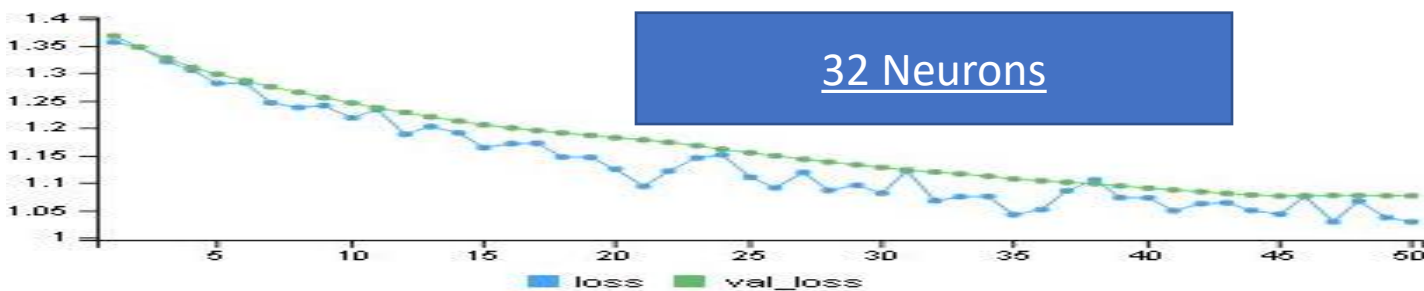
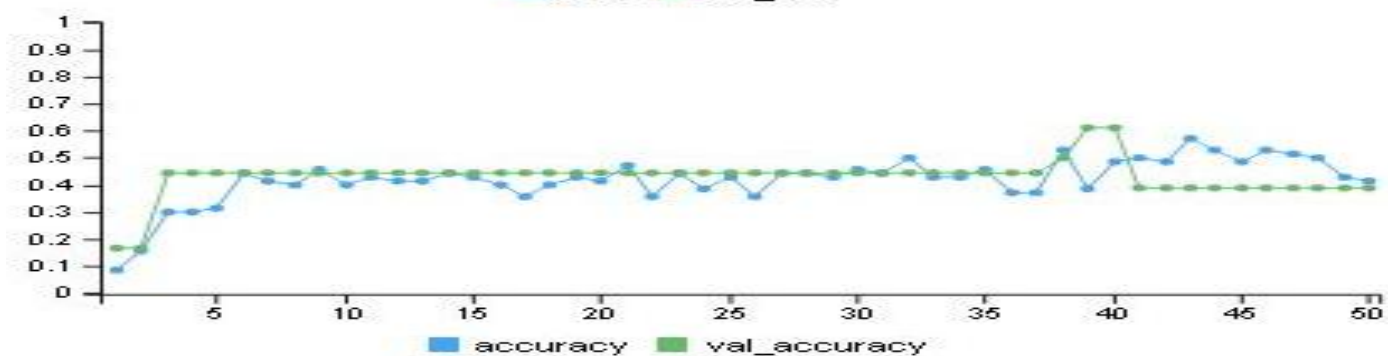
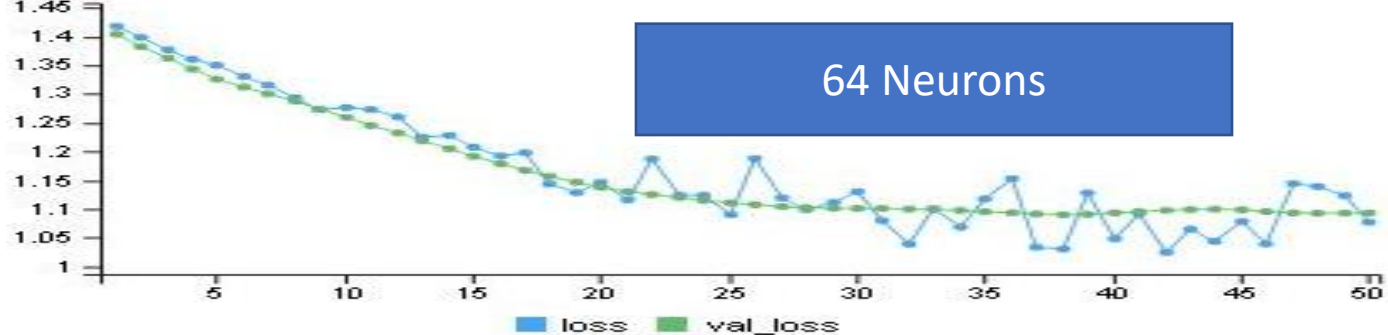
- runs=tuning_run ("experiment1.R", flags=list (dense_units=c (32,64)))
- multi-layer perceptron of multi-class classification of student different grades.

```
FLAGS= flags(flag_integer('dense_units',32))           #hidden layer 32
model =keras_model_sequential() %>%
  layer_dense(units=FLAGS$dense_units,
  activation = 'relu',
  input_shape = c(13)) %>%                             #21 independent variables
  layer_dropout(rate=0.1) %>%                             # 0.1for avoiding 10 %overfitting
  layer_dense(units=16                                   # 16 another dense layer
  activation = 'relu') %>%
  layer_dropout(rate=0.01) %>%                             #0.1
  layer_dense(units=4,activation='softmax') # our response is A,A-,B, B-..... units layers
#compile
model %>% compile(loss='categorical_crossentropy'      ,# for more than two layers
  optimizer='adam', metrics='accuracy')
```

Here researcher may use **stochastic gradient descent adms and rmsprop algorithm for fine tune.**

The loss function depends for a regression problem or mean squared error selection.

```
history=model %>% fit (training , trainlabels , epochs=50, batch_size=32, validation split=0.2)
```



```
head(runs)
```

```
Data frame: 2 x 23
```

```
run_dir metric_loss metric_accuracy
```

```
metric_val_loss metric_val_accuracy
```

```
1 runs/2020-04-20T05-46-01Z 1.0343
```

```
0.5714 1.0767 0.4444
```

```
2 runs/2020-04-20T05-45-15Z 1.0661
```

```
0.4857 1.1183 0.5000
```

```
> runs[,c(5,6)]
```

```
Data frame: 2 x 2
```

```
metric_val_accuracy flag_dense_units
```

```
1 0.4444 64
```

```
2 0.5000 32
```

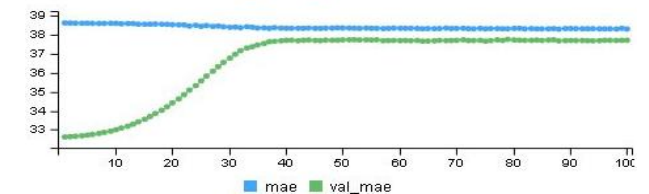
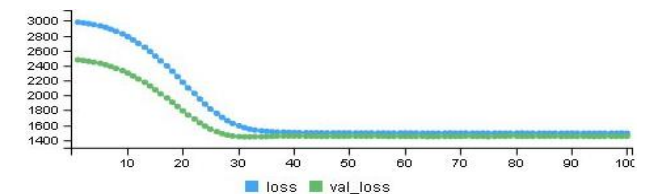
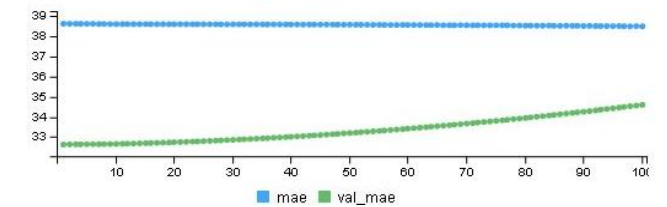
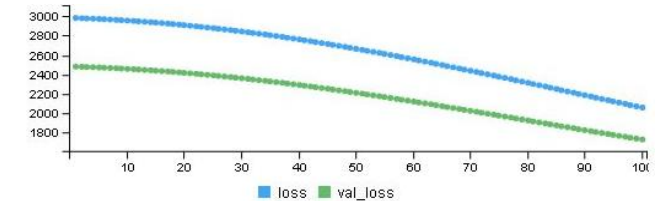
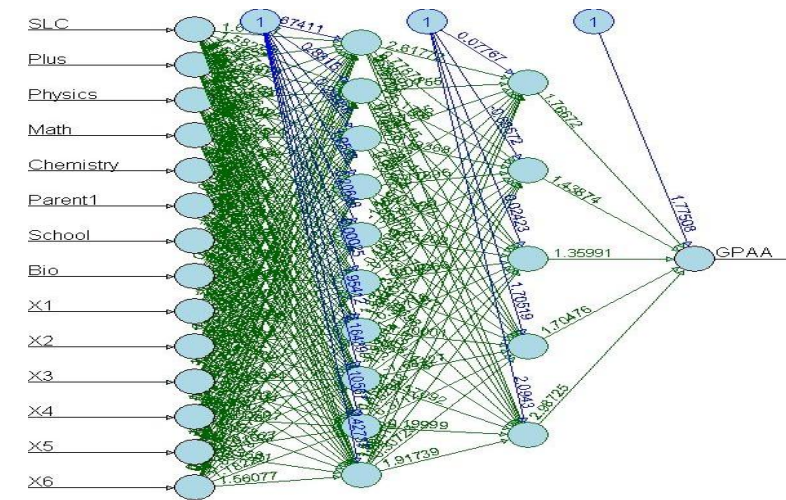
Deep Learning Full Model

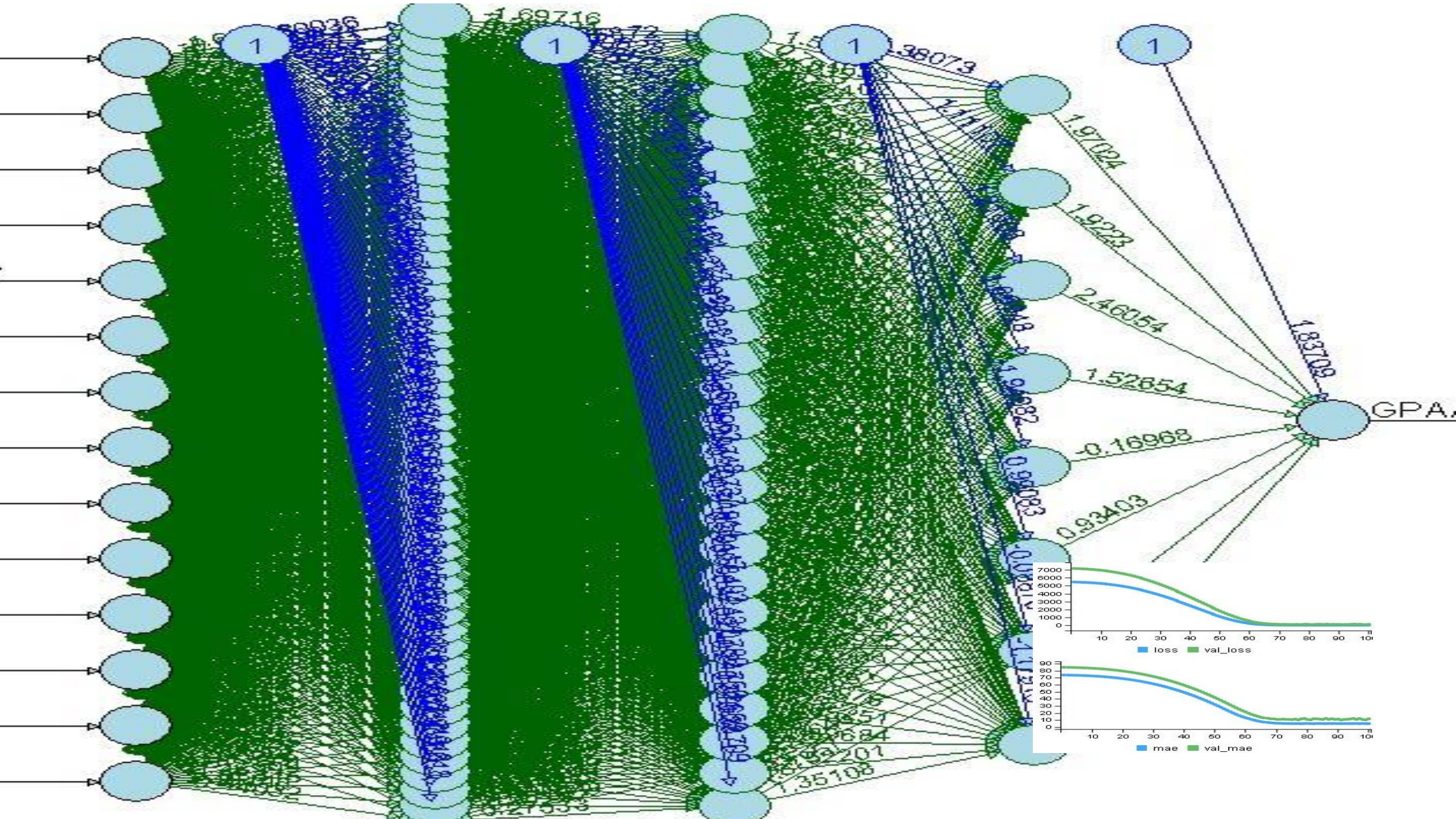
After loading **keras**, **mlbench**, **dplyr**, **MASS**, **magrittr** and neural network in r console library the set of student records is read using read.csv file and converted into its data frame.

after converting data set into **data\$X6=as.numeric(data\$X6)** numeric of all 14 independent and dependent variables using tidyverse and filter command `ss=data %>%select (GPAA, SLC, School, Plus, Physics, Math, Chemistry, Bio, Parent1, X1, X2, X3, X4, X5, X6)` the command `data%<>%mutate_if(is.factor,as.numeric)#converting all factor variables into numeric` pattern of these selected variables.

The neural network model of GPAA categorical variables of different GPA grades of **580 student** records with having 10,5 hidden layers in-between 14 independent variables and GPA variables using `>n=neuralnet (GPAA~ SLC+ Plus+ Physics+ Math +Chemistry +Parent1+School+Bio+X1+X2+X3+X4+X5+X6, data=data, hidden=c(10,5), linear. output=F,lifesign='full',rep=1)`

will conversed in **25 steps** with **error: 858.00513** in **time 0.05 secs.**





MLP using fine tune improvement (100,50,20,7)

The more layers hidden layer multiple perception using deep neural network use 100,50,20 and 7 layers is again tested using `model=keras_model_sequential()`

```
model%>%
```

```
layer_dense(units=100,activation = 'relu',input_shape =c(14)) %>%
```

```
layer_dropout(rate=0.4) %>% # 40 % neurons were being ignored while mode
```

```
layer_dense(units=50,activation = 'relu') %>%
```

```
layer_dropout(rate=0.3) %>%
```

```
layer_dense(units=20,activation = 'relu') %>%
```

```
layer_dropout(rate=0.2) %>%
```

```
layer_dense(units=7)#default activation Linear activation
```

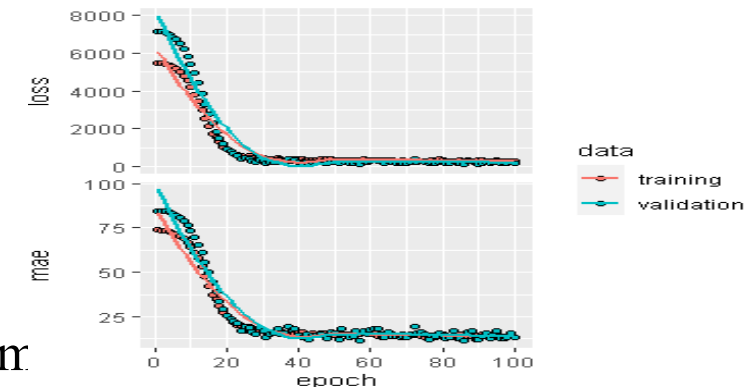
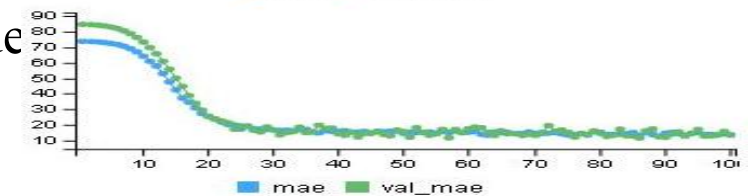
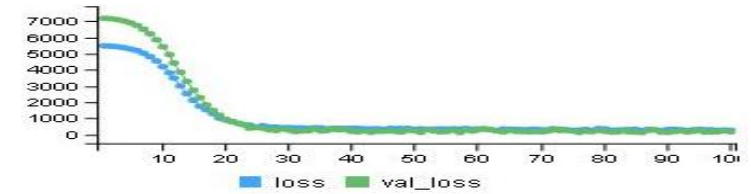
```
dense_16 (Dense) (None, 7)
```

147

Total params of this model is 7,717 and `summary(model)` describes.

The model compile there is regression problem using `model %>% compile(loss='r`
`optimizer=optimizer_rmsprop(lr=0.001), metrics='mae')` mean absolute error. The
`history=model %>% fit(training,trainingtarget, epochs=100, batch_size=32, validation_split=0.2)` produces as
Epoch 99/100 6/6 [=] - 0s 76ms/step - loss: 295.4308 - mae: 14.0018 - val_loss: 251.4277 - val_mae:

15.6809.Epoch 100/100 6/6 [=] - 0s 74ms/step - loss: 279.2279 - mae: 13.2599 - val_loss: 187.8508 - val_mae:
13.5024



multiple layers with five neuron produces 72 percent accuracy results.

Performance Measures

Predicator Variable	No of Input	No of Hidden	Training accuracy	Test Accuracy
P/F	22 Full	1	91	81
P/F	22 Half	1	98	95
GPA	13 Full	1	83	86
GPA	13 Half	1	99	82
GPA	14 Full	7,3,2	83	83
GPA	14 Full	7,3,2 B	90	90
GPA	13 Full	25,12,6,3	82	82
Multilayer Perception Deep Neural Network				
Predicator Variable	No of Input	No Hidden	Model Loss	Mean Accuracy Error
GPA(Marks)	Full	(10,8)	2086.56	39.04
GPA(Marks)	Full	(50,25,8)	1456.60	37.85
GPA(Marks)	Full	(100,50,20,8)	3021.10	39.79
GPA(Marks)	Half	(10/7)	4571.20	67.30
GPA(Marks)	Half	(50,25,7)	55.24	6.44
GPA(Marks)	Half	(100,50,20,7)	72.11	7.47

What is Machine Learning

- “Learning is process by which a system improves performance **from the experiences.**” - Herbert Simon
- Machine learning is an application of artificial intelligence **(AI)** that provides the ability to **automatically learn** and improve prediction from past experiences without being explicitly programmed.
- Machine Learning is the study of algorithms that improve their performance P at some task T with experience E I.e is learning.

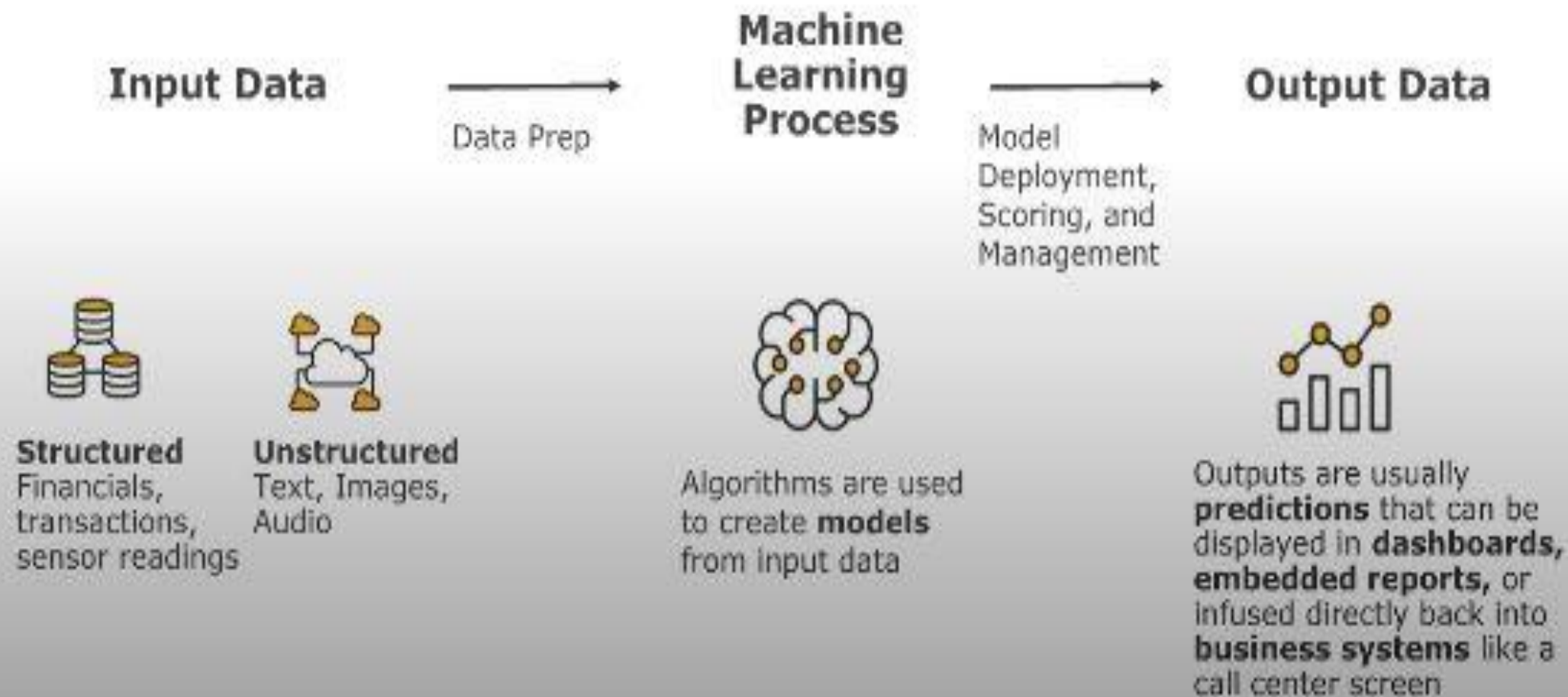


Differentiate between ML and Traditional Programming

Machine Learning



How does data science work?



Some Application of Machine learning

Categorizing News



BUSINESS & ECONOMY
Paying service charge at hotels not mandatory



TECHNOLOGY & SCIENCE
The 'dangers' of being admin of a WhatsApp group



ENTERTAINMENT
This actor stars in Raabta. Guess who?



IPL 2017
Preview: Bullish KKR face depleted Lions



INDIA
Why is Aadhaar mandatory for PAN? SC asks Centre

Email Spam Detection



Face Recognition



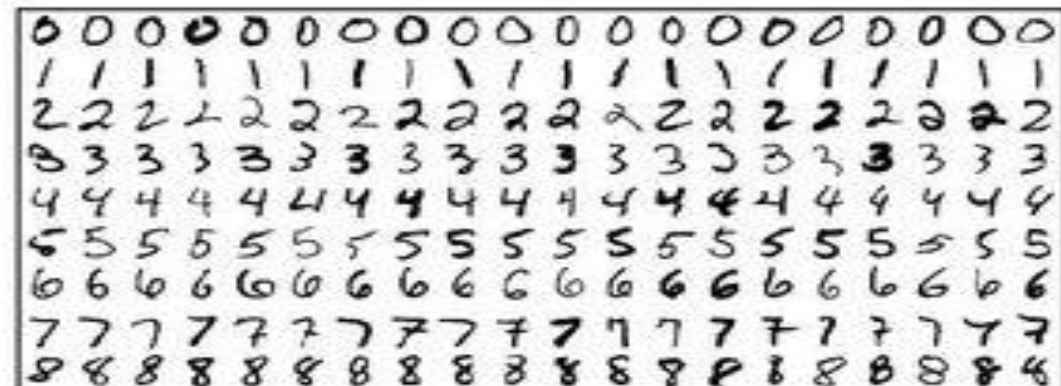
Sentiment Analysis



Medical Diagnosis

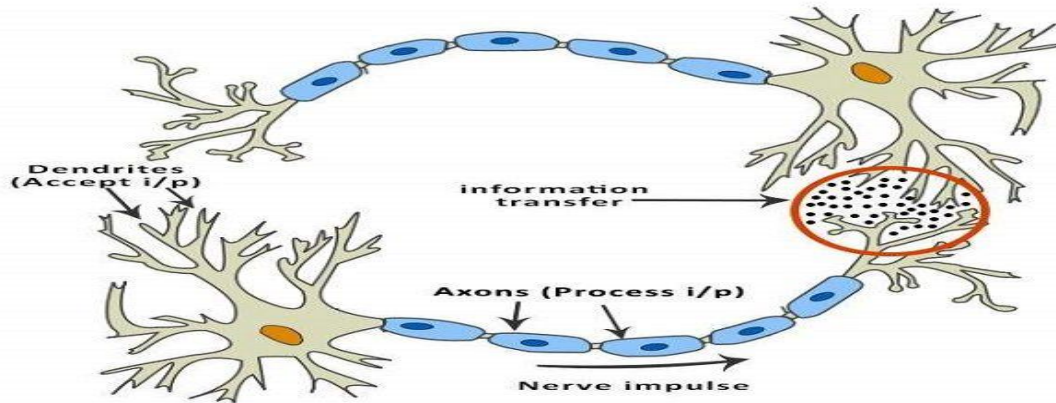


Digit Recognition



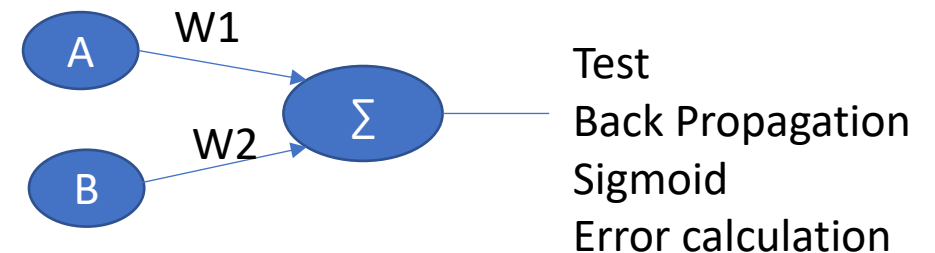
What is Artificial Neural Network

- The human brain is composed of **86 billion** nerve cells called **neurons**. (Herculano-Houzel - 2009)
- They are connected to other thousand neuron cells by **Axons**.
- Stimuli from external environment or inputs from sensory organs are accepted by dendrites.
- These inputs create electric impulses, which quickly travel through the neural network.
- A neuron can then send the message to other neuron to handle the issue or does not send it forward.
- When one neuron connected with another neuron there is some weights.



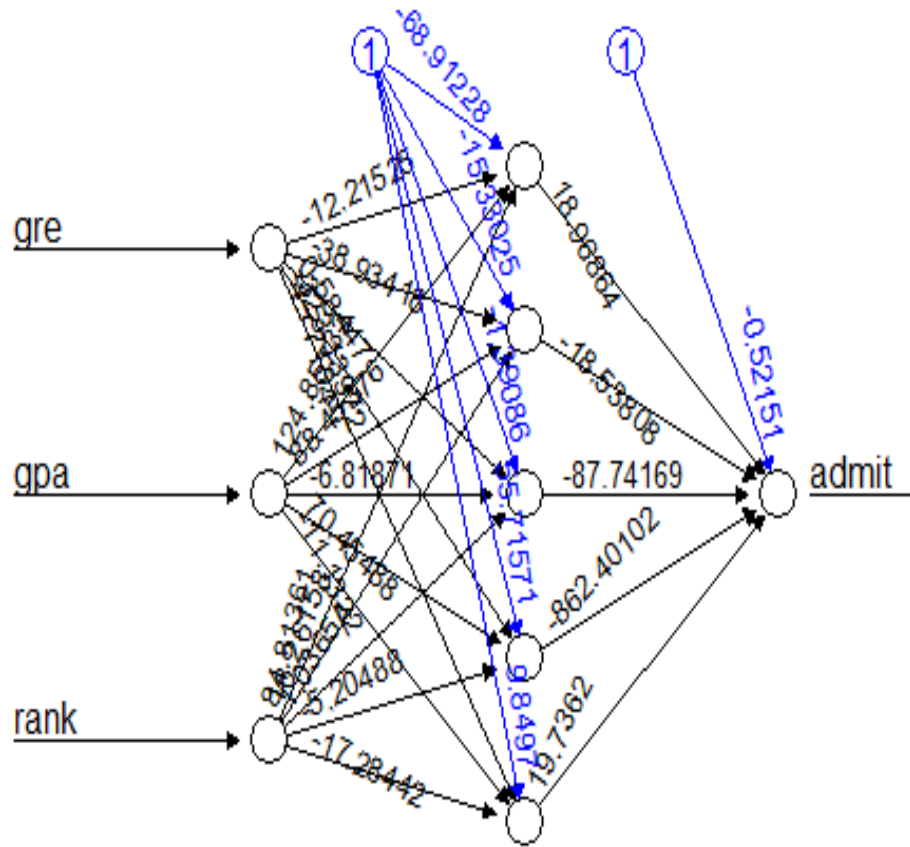
Dendrites (input) Soma (process) Axons(produce output)
Synapsis (Electronic chemical signals)

This concept was developed by BERNARD WISROW of Stanford University in 1950

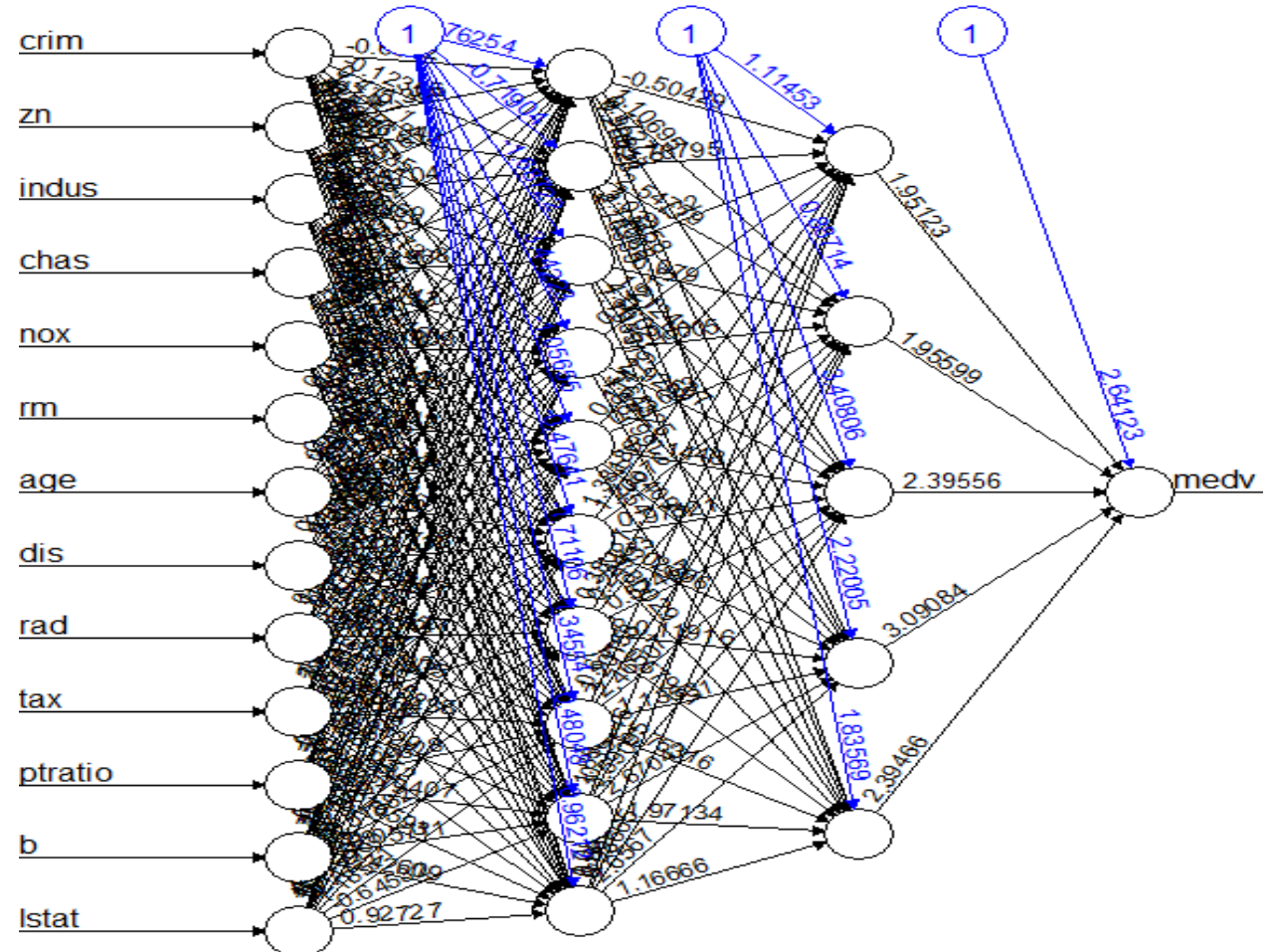


Advantage and disadvantages

Disadvantage	Advantages
Long time for training data	Applicable for Noisy data



Error: 132.550704 Steps: 18339

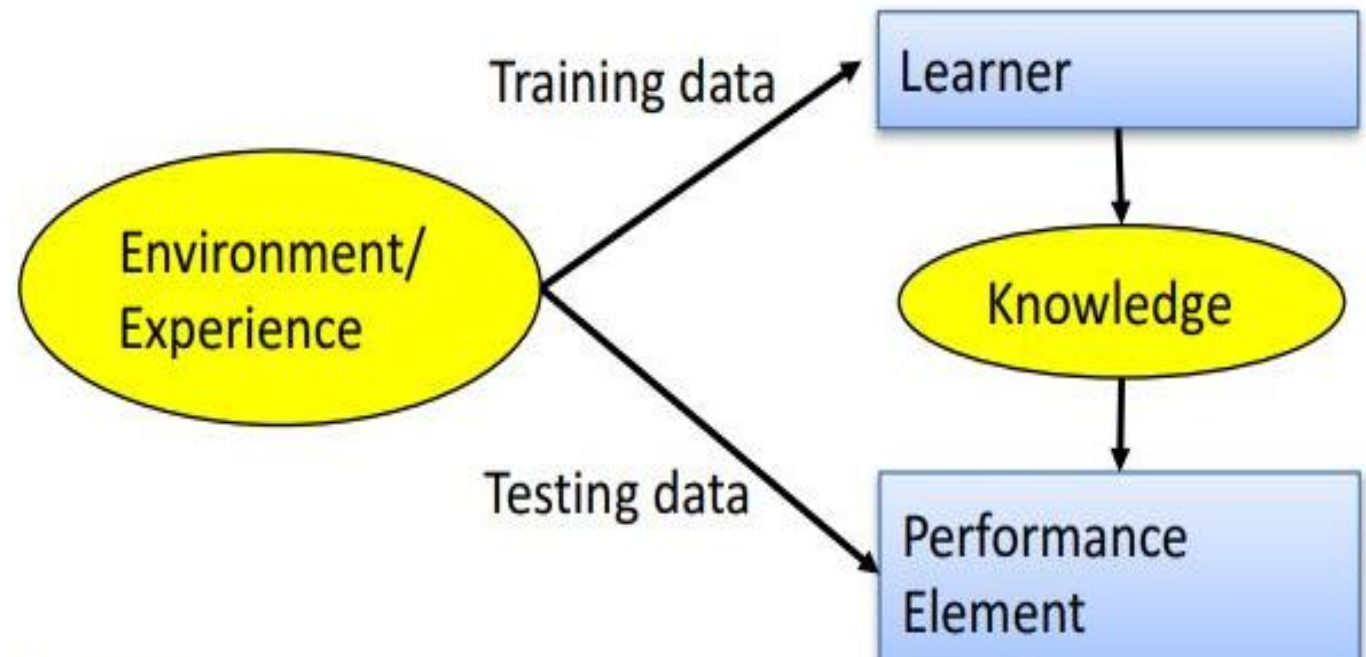


Types of Learning

- Supervised (inductive) learning
 - Given: training data + **desired outputs** (labels) for Classification
- Unsupervised learning
 - Given: training data (**without desired outputs**) Clustering
- Semi-supervised learning
 - Given: training data + a few desired outputs

Designing a Learning System

- Choose the training experience
- Choose exactly what is to be learned – i.e. the target function
- Choose how to represent the target function
- Choose a learning algorithm to infer the target function from the experience



Neural Network (NN): A **neural network** is a computational model inspired by the way biological neural networks in the human brain work. It consists of layers of **nodes (neurons)**, each connected to others in the next layer. Neural networks are used for tasks like classification, regression, and pattern recognition.

Key Components:

- **Input layer:** Receives the input data.
- **Hidden layers:** Process inputs using weighted connections and activation functions.
- **Output layer:** Produces the final result.
- **Weights and biases:** Parameters adjusted during training.
- **Activation functions:** Introduce non-linearity (e.g., ReLU, Sigmoid).

Example: A neural network might be used to classify whether an email is spam or not based on features like subject line, content, and sender

Deep Learning

Deep Learning is a subfield of machine learning that uses **neural networks with many layers**—hence the word "deep". These are called **Deep Neural Networks (DNNs)**. Deep learning models can automatically learn complex patterns in large datasets.

Common Architectures:

- **Convolutional Neural Networks (CNNs)**: Great for image processing.
- **Recurrent Neural Networks (RNNs)**: Used for sequence data like time series and text.
- **Transformers**: Powerful models for language tasks (e.g., ChatGPT is based on one).
- **Autoencoders**: Used for unsupervised learning and dimensionality reduction.
- **GANs (Generative Adversarial Networks)**: Generate new data samples, like deepfakes or art.

Feature	Neural Network (NN)	Deep Learning (DL)
Layers	Usually 1–2 hidden layers	Many hidden layers (deep)
Data Requirements	Works with small to medium data	Needs large datasets
Feature Engineering	Often manual	Learns features automatically
Computation Power	Less demanding	Requires GPUs/TPUs for efficiency

Perceptron: A **Perceptron** is the **simplest form of a neural network**, used for **binary classification of linearly separable data**.

Algorithm:

1. **Initialize weights** and bias randomly.
2. For each training sample (x, y):
 - Compute output:

$$y_{\text{pred}} = \begin{cases} 1, & \text{if } w \cdot x + b > 0 \\ 0, & \text{otherwise} \end{cases}$$

- Update weights:

$$w := w + \eta(y_{\text{true}} - y_{\text{pred}})x$$

$$b := b + \eta(y_{\text{true}} - y_{\text{pred}})$$

- Repeat for multiple epochs.

Benefits:

- Simple to implement.
- Works well for **linearly separable** data.

Limitations:

- Cannot handle **non-linear** decision boundaries (e.g., XOR problem).
- Only handles **binary classification**.

Perceptron Learning – | Suggested: Perceptron Rule to design XOR Logic

- The first neural network model 'Perceptron', designed in 1958
- Perceptron is a linear binary classifier used for supervised learning.
- Perceptron Learning model is a combination of two concepts, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights.

Perceptron Learning – Algorithm

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:
$$Y = \text{Step}((x_1 w_1 + x_2 w_2) - \theta)$$
3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} .

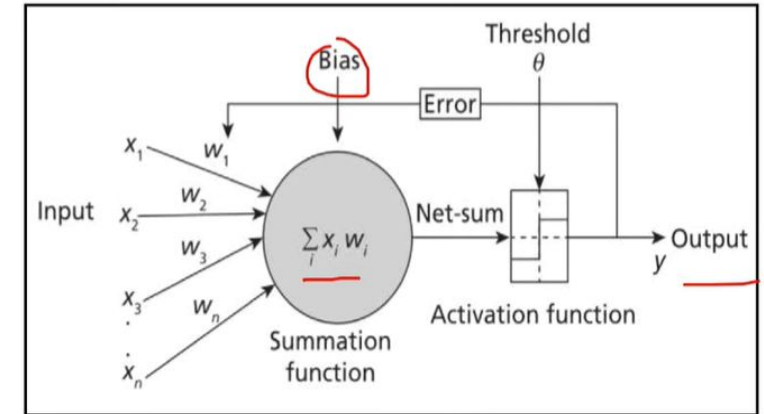
$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

Perceptron Learning – Introduction

The perceptron model consists of 4 steps:

1. Inputs from other neurons
2. Weights and bias
3. Net sum
4. Activation function

$$Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}$$



5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$
$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

- In case of the perceptron learning rule, the learning signal is the difference between the calculated output and actual (target) output of a neuron.
- The output “y” is obtained on the basis of the net input calculated and activation function being applied over the net input.

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

If $y \neq t$, then

$$w(\text{new}) = w(\text{old}) + \alpha t x \quad (\alpha - \text{learning rate})$$

else, we have

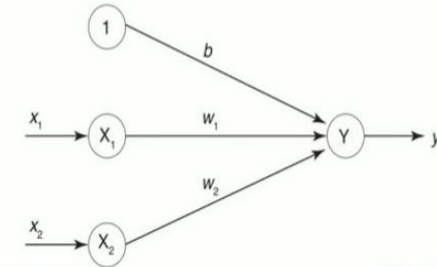
$$w(\text{new}) = w(\text{old})$$

- Weights are updated using the formula

AND function using Perceptron Rule Solved Example

- The perceptron network, which uses perceptron learning rule, is used to train the AND function.
- The input patterns are presented to the network one by one.
- When all the four input patterns are presented, then one epoch is said to be completed.
- The initial weights and threshold are set to zero.
- The learning rate α is set equal to 1.

x_1	x_2	t
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1



AND function using Perceptron Rule Solved Example

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

$1 + 1 + (-1) = 1$
 $0 + 1 \times 0 + 1 \times 0 = 0$

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

$$\Delta w_1 = \alpha t x_1;$$

$$\Delta w_2 = \alpha t x_2;$$

$$\Delta b = \alpha t$$

$\alpha = 1$



Input		Target (t)	Net input (y_{in})	Calculated output (y)	Weight changes			Weights		
x_1	x_2				Δw_1	Δw_2	Δb	w_1	w_2	b
								(0)	(0)	(0)
EPOCH-1										
1	1	1	0	0	1	1	1	1	1	1
1	-1	-1	1	1	-1	1	-1	0	2	0
-1	1	-1	2	1	+1	-1	-1	1	1	-1
-1	-1	-1	-3	-1	0	0	0	1	1	-1

Given

Y and y' same not update other wise UpToDate

2. Multi-Layer Perceptron (MLP)

An **MLP** is a **feedforward neural network** consisting of:

- **Input layer**
- One or more **hidden layers**
- **Output layer**

Each layer is made of **neurons (perceptrons)** with **non-linear activation functions**.

Algorithm (Training Process):

A. Forward Propagation:

For each layer:

$$z = W \cdot x + b$$

$$a = \text{activation}(z)$$

- W = weights, b = biases, a = activated output

B. Loss Calculation:

- Common loss functions: Mean Squared Error (MSE), Cross-Entropy

C. Backpropagation (Gradient Calculation):

- Compute gradients using the **chain rule** to determine how weights influence the loss.

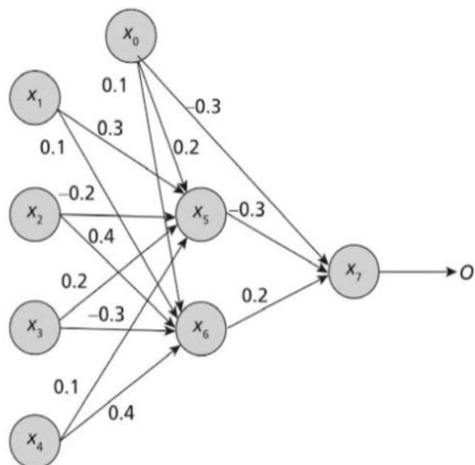
D. Weight Update (Gradient Descent):

Repeat forward + backpropagation over many **epochs**.

$$w := w - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

Multi-Layer Perceptron Learning – Solved Example

- The given MLP consists of an Input layer, one Hidden layer and an Output layer.
- The input layer has 4 neurons, the hidden layer has 2 neurons and the output layer has a single neuron.
- Train the MLP by updating the weights and biases in the network.
- Learning Rate = 0.8

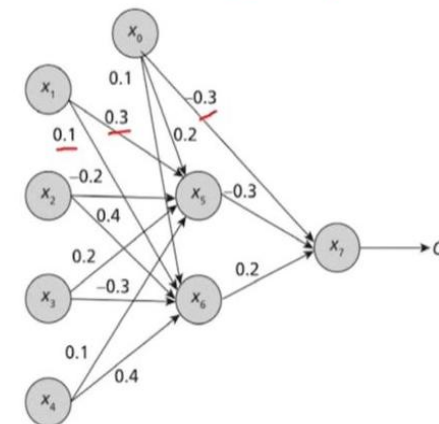


Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

- Calculate Input and Output in the Input Layer
Net Input and Output Calculation

Input layer	I_j	O_j
x_1	1	1
x_2	1	1
x_3	0	0
x_4	1	1



Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

- Calculate Net Input and Output in the Hidden Layer and Output Layer as

Unit _j	Net Input I_j	Output O_j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.3 + 1 \times -0.2 + 0 \times 0.2 + 1 \times 0.1 + 1 \times 0.2 = 0.4$	$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-0.4}} = 0.599$
x_6	$I_6 = x_1 \times w_{16} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_6 = 1 \times 0.3 + 1 \times 0.4 + 0 \times -0.3 + 1 \times 0.4 + 1 \times 0.1 = 1.2$	$O_6 = \frac{1}{1 + e^{-I_6}} = \frac{1}{1 + e^{-1.2}} = 0.769$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.599 \times -0.3 + 0.769 \times 0.2 + 1 \times -0.3 = -0.326$	$O_7 = \frac{1}{1 + e^{-I_7}} = \frac{1}{1 + e^{-0.326}} = 0.419$

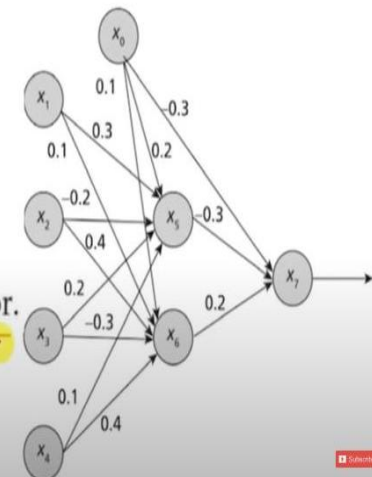
- Calculate Error = $O_{desired} - O_{Estimated}$

So, error for this network is:

$$\text{Error} = O_{desired} - O_7 = 1 - 0.419 = 0.581$$

So, we need to back propagate to reduce the error.

If ok



Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

Error Calculation for Each Unit in the Output Layer and Hidden Layer

For Output Layer Unit _k	Error _k
x_7	$\text{Error}_7 = O_7(1 - O_7)(Y_n - O_7)$ $= 0.419 \times (1 - 0.419) \times (1 - 0.419) = 0.141$
For Hidden Layer Unit _j	Error _j
x_6	$\text{Error}_6 = O_6(1 - O_6) \sum_k \text{Error}_k w_{jk} = O_6(1 - O_6) \text{Error}_7 w_{67}$ $= 0.769(1 - 0.769) \times 0.2 \times 0.141 = 0.005$
x_5	$\text{Error}_5 = O_5(1 - O_5) \sum_k \text{Error}_k w_{jk} = O_5(1 - O_5) \text{Error}_7 w_{57}$ $= 0.599(1 - 0.599) \times 0.141 \times -0.3 = -0.0101$

Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

Update bias using the below formula:

$$\theta_j = \theta_j + \alpha \times \text{Error}_j$$

θ_j	$\theta_j = \theta_j + \alpha \times \text{Error}_j$	New Bias
θ_5	$\theta_5 = \theta_5 + \alpha \times \text{Error}_5$ $= 0.2 + 0.8 \times -0.0101$	0.192
θ_6	$\theta_6 = \theta_6 + \alpha \times \text{Error}_6$ $= 0.1 + 0.8 \times 0.005$	0.104
θ_7	$\theta_7 = \theta_7 + \alpha \times \text{Error}_7$ $= -0.3 + 0.8 \times 0.141$	-0.187

Otherwise weight adjustment

Multi-Layer Perceptron Learning – Solved Example

2. Calculate Net Input and Output in the Hidden Layer and Output Layer

Net Input and Output Calculation in the Hidden Layer and Output Layer

Unit j	Net Input I _j	Output O _j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.292 + 1 \times -0.208 + 0 \times 0.2 + 1 \times 0.092 + 1 \times 0.192 = 0.368$	$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-0.368}} = 0.591$
x_6	$I_6 = x_1 \times w_{16} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_6 = 1 \times 0.292 + 1 \times 0.404 + 0 \times -0.3 + 1 \times 0.404 + 1 \times 0.104 = 1.204$	$O_6 = \frac{1}{1 + e^{-I_6}} = \frac{1}{1 + e^{-1.204}} = 0.7692$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.591 \times -0.232 + 0.7692 \times 0.287 + 1 \times -0.187 = -0.326$	$O_7 = \frac{1}{1 + e^{-I_7}} = \frac{1}{1 + e^{-0.326}} = 0.474$

Happy st
otherwi
repeate
the

Multi-Layer Perceptron Learning – Solved Example

- The output we receive in the network at node 7 is 0.474

$$\text{Error} = 1 - 0.474 = 0.526$$

- Now, when we compare the error, we get in the previous iteration and in the current iteration,

$$\text{Error reduced is } 0.581 - 0.526 = 0.055$$

Benefits:

- Can model **non-linear** patterns.
- Works for **classification** and **regression**.
- Backbone of deep learning.

Limitations:

- Requires **more data** and **tuning**.
- May **overfit** on small datasets.
- Needs **activation functions** like ReLU or Sigmoid to introduce non-linearity.

$$w := w - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

Steps:

1. Forward pass

2. Compute loss.

3. Backward pass:

1. Compute gradients of the loss with respect to each weight.
2. Use the **chain rule** to propagate the error backward.

4. Update weights using:

3. Backpropagation Backpropagation is the **learning algorithm** used to **train MLPs**. It calculates **how much each neuron contributed to the error**, then adjusts weights to reduce the error.

Benefits:

- Efficient training of deep networks.
- Enables MLPs to learn complex mapping

Optimization Methods:

- Stochastic Gradient Descent (SGD)
- Adam, RMSProp, Momentum

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Output range: (0, 1)

Shape:

- S-shaped curve (smooth and non-linear).

Used for:

- Binary classification (e.g., final layer in binary logistic regression).
- Historically used in hidden layers (less common now).

Pros:

- Smooth gradient.
- Output is easy to interpret as probability.



2. ReLU (Rectified Linear Unit)

Formula:

$$f(x) = \max(0, x)$$

Output range: [0, ∞)

Shape:

- 0 for negative inputs, linear for positive inputs.

Used for:

- Hidden layers in almost all modern deep neural networks (CNNs, RNNs, etc.).

Pros:

- Computationally efficient.
- **Avoids vanishing gradient** for positive inputs.
- Sparse activation (many neurons are "off", making the model efficient).

Model Type	Description
Single-layer Perceptron	One layer (only solves linearly separable problems)
Multi-layer Perceptron (MLP)	Multiple layers, handles non-linearity
Deep Neural Network	MLP with many hidden layers (deep learning)

Feature	Perceptron	MLP	Backpropagation
Type	Linear	Non-linear	Learning algorithm
Layers	1	≥2	Used to train MLPs
Activation Function	Step	ReLU, Sigmoid	Depends on model
Can solve XOR problem?	✗	✓	✓
Handles non-linearity	✗	✓	✓

Types of MLP Networks

MLPs are typically **fully connected**, but based on the number of layers and tasks, we can classify them as:

Type	Description
Shallow MLP	1 hidden layer
Deep MLP	Multiple hidden layers
Regression MLP	Outputs continuous values
Classification MLP	Outputs class probabilities
Autoencoders	MLPs trained to reconstruct inputs

Type	Description
Batch GD	Uses all training samples before updating weights
Stochastic GD (SGD)	Updates weights after each sample
Mini-batch GD	Updates weights after a small batch of samples (best trade-off)

Applications of Deep Learning:

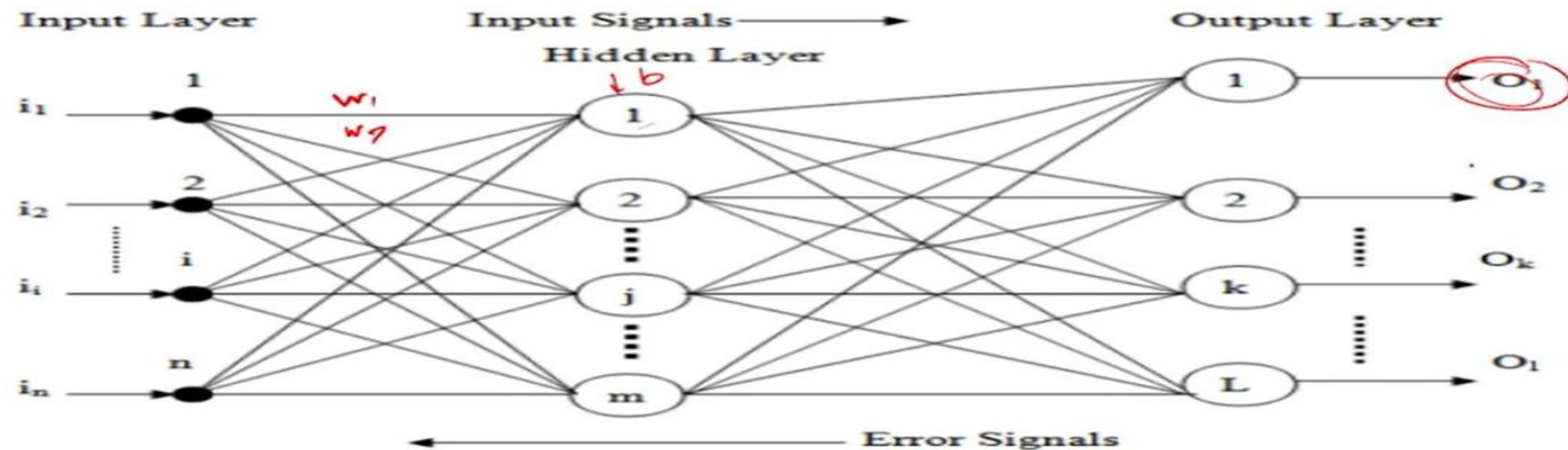
- Image recognition (e.g., face detection)
- Speech recognition (e.g., Siri, Alexa)
- Natural language processing (e.g., translation, chatbots)
- Autonomous vehicles
- Healthcare (e.g., tumor detection)

Back Propagation Neural Network



- Output are compared with correct output to compute value of some predefined error function.
- Using this information, Algorithms adjust weights of each connection in order to reduce value of error function by some small amount.
- After repeating this process for a sufficiently large number of training cycles the network will usually converge to some state where error of the calculation is small.
- The goal of back propagation, as with most training algorithms, is to iteratively adjust the weights in network to produce the desired output by minimizing the output error.
- It is gradient descent approach (use minimization of first-order derivatives to find optimal solution).
- To propagate error signals, start at the output layer & work backward to the hidden layer.

Back Propagation Neural Network



Previous weight
adjustment

1. CNN – Convolutional Neural Networks

CNNs are a type of deep learning model that can capture spatial hierarchies in data.

Component

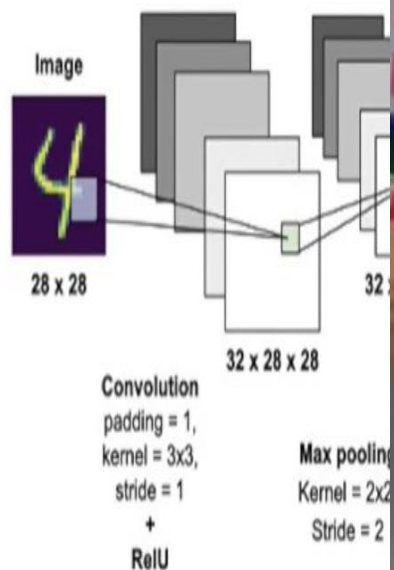
Convolutional Layer

Activation (ReLU)

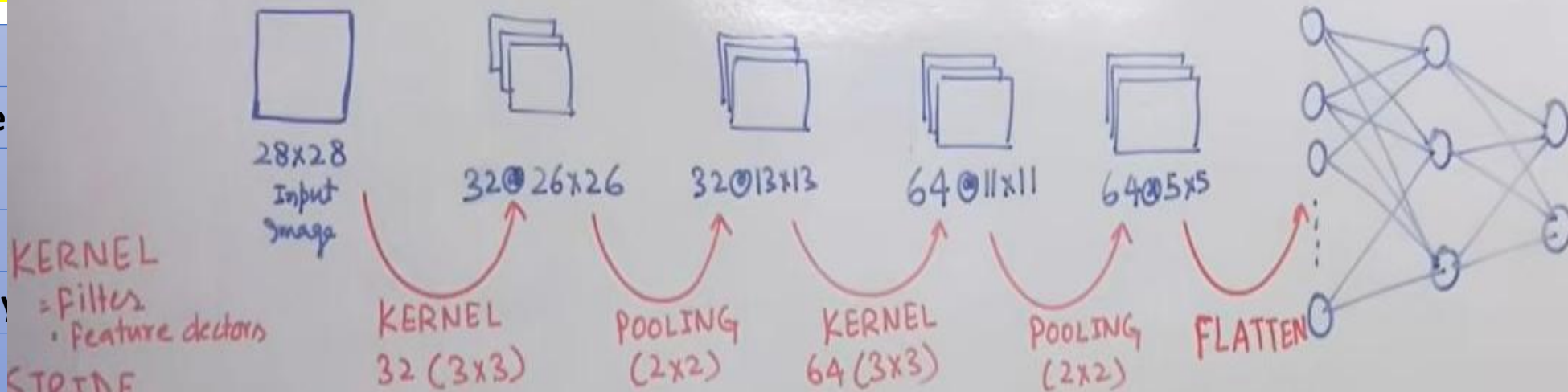
Pooling Layer

Fully Connected Layer

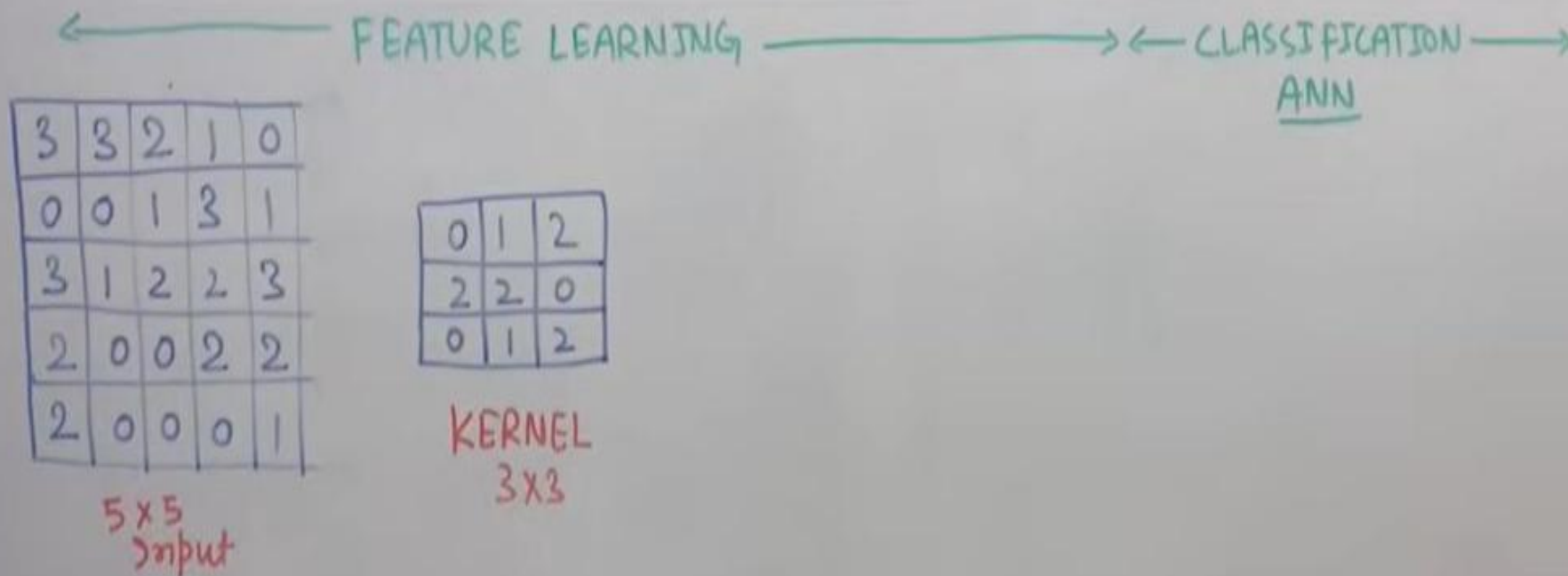
Softmax Layer



CONVOLUTIONAL NEURAL NETWORKS : BUILDING BLOCKS



<https://meet.google.com/gag-yatm-udz>



Filter-Kernel



Mathematical Form

1	0	9	3	0
3	6	0	0	5
5	3	8	2	0
1	5	7	9	7
4	8	2	2	0

5 × 5 image

0	1	0
3	2	1
1	3	2

3 × 3 Kernel

$$1 \times 0 + 0 \times 1 + 9 \times 0 + 3 \times 3 + 6 \times 2 + 0 \times 1 + 5 \times 1 + 3 \times 3 + 8 \times 2$$

51		

Filter-Kernel



Mathematical Form

1	0	9	3	0
3	6	0	0	5
5	3	8	2	0
1	5	7	9	7
4	8	2	2	0

5 × 5 image

0	1	0
3	2	1
1	3	2

3 × 3 Kernel

$$8 \times 0 + 2 \times 1 + 0 \times 0 + 7 \times 3 + 9 \times 2 + 7 \times 1 + 2 \times 1 + 2 \times 3 + 0 \times 2$$

51	58	22
65	74	77
55	64	56

To handle **variety** in digits we can use simple artificial neural network (ANN)



1	1	1	-1	-1
1	-1	1	-1	-1
1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1
1	-1	-1	-1	-1



1	1	1	-1	-1
1	1	1	-1	-1
1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1
1	-1	-1	-1	-1



-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

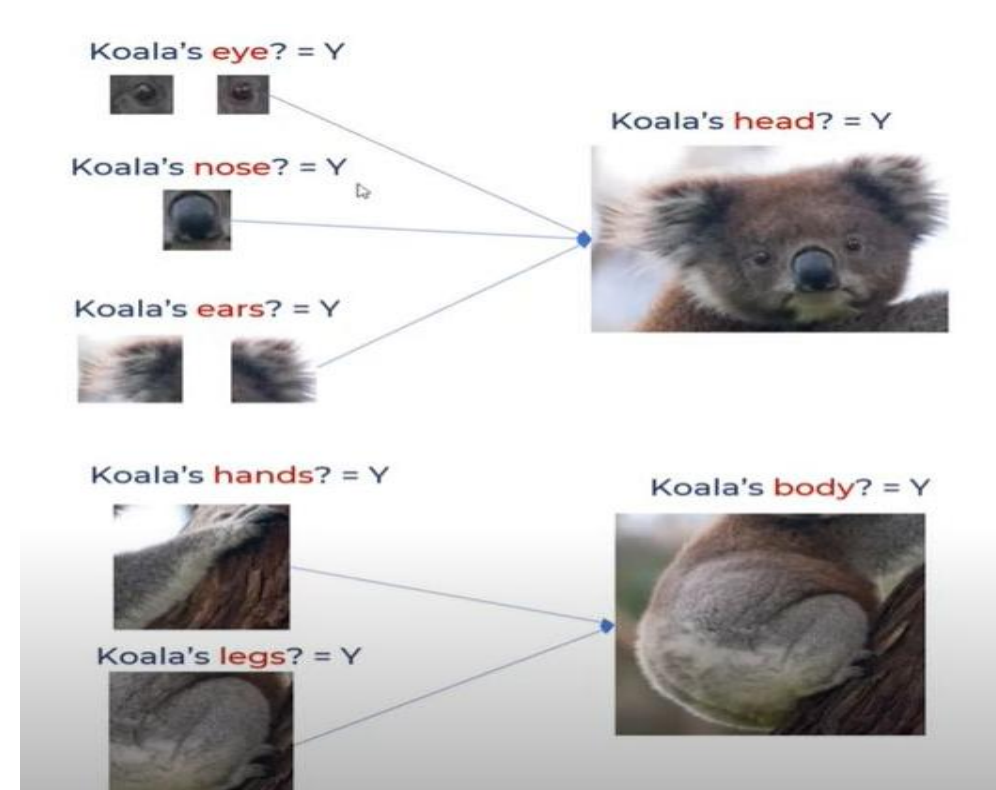
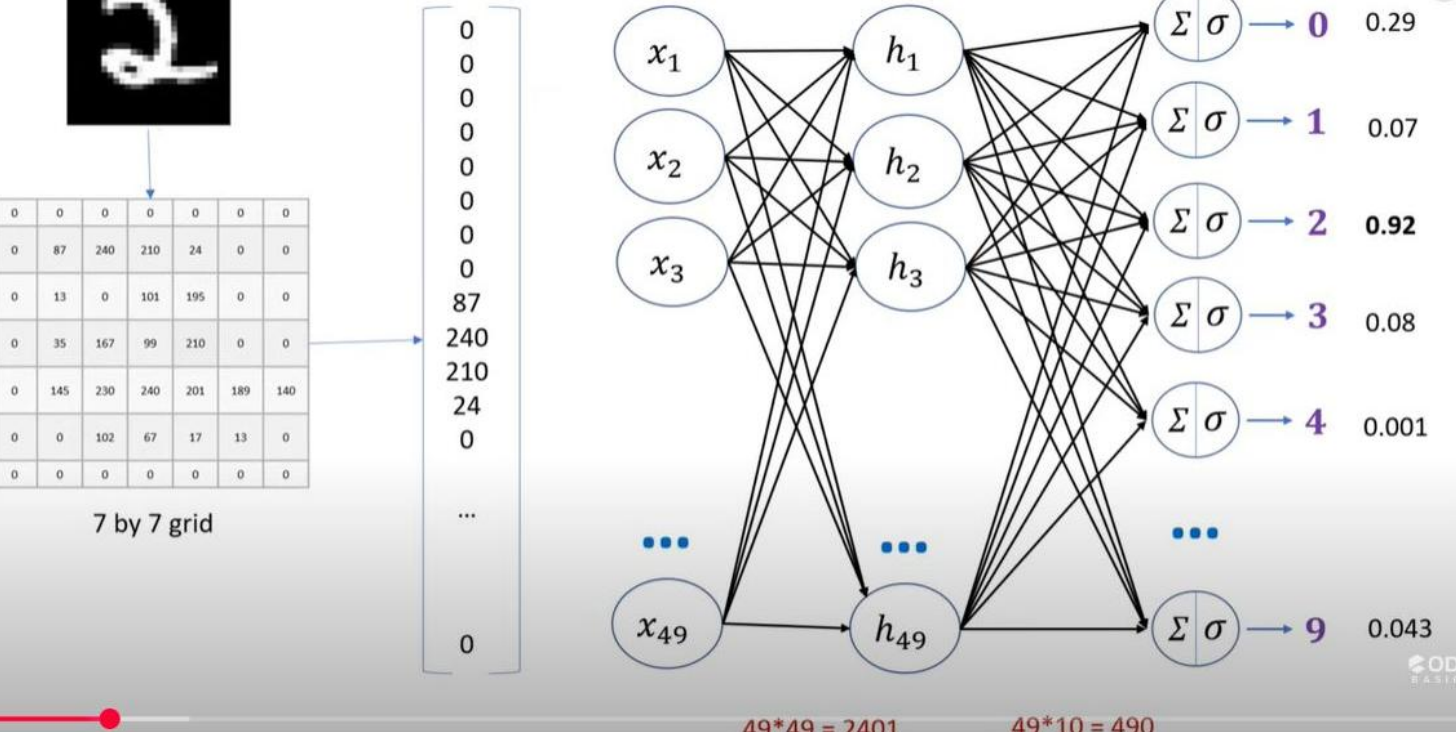


Image size = 1920 x 1080 X 3

First layer neurons = 1920 x 1080 X 3 ~ 6 million

Hidden layer neurons = Let's say you keep it ~ 4 million

Weights between input and hidden layer = 6 mil * 4 mil
= 24 million



$$-1+1+1-1-1-1-1+1+1 = -1 \rightarrow -1/9 = -0.11$$

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11		

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33

Feature Map



eye *

1	1
1	1

 =

1	1
1	1



nose *

1
1

 =

1
1



ears *

1	1
1	1

 =

1	1
1	1



hands *

1
1

 =

1
1



legs *

1
1

 =

1
1



*

*



flatten



flatten



-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

→

-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



→

0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

9

*

1	1	1
1	-1	1
1	1	1

=

	1	

Loopy pattern detector

8

*

1	1	1
1	-1	1
1	1	1

=

	1	
	1	

Loopy pattern detector

6

*

1	1	1
1	-1	1
1	1	1

=

	1	

Loopy pattern detector

96

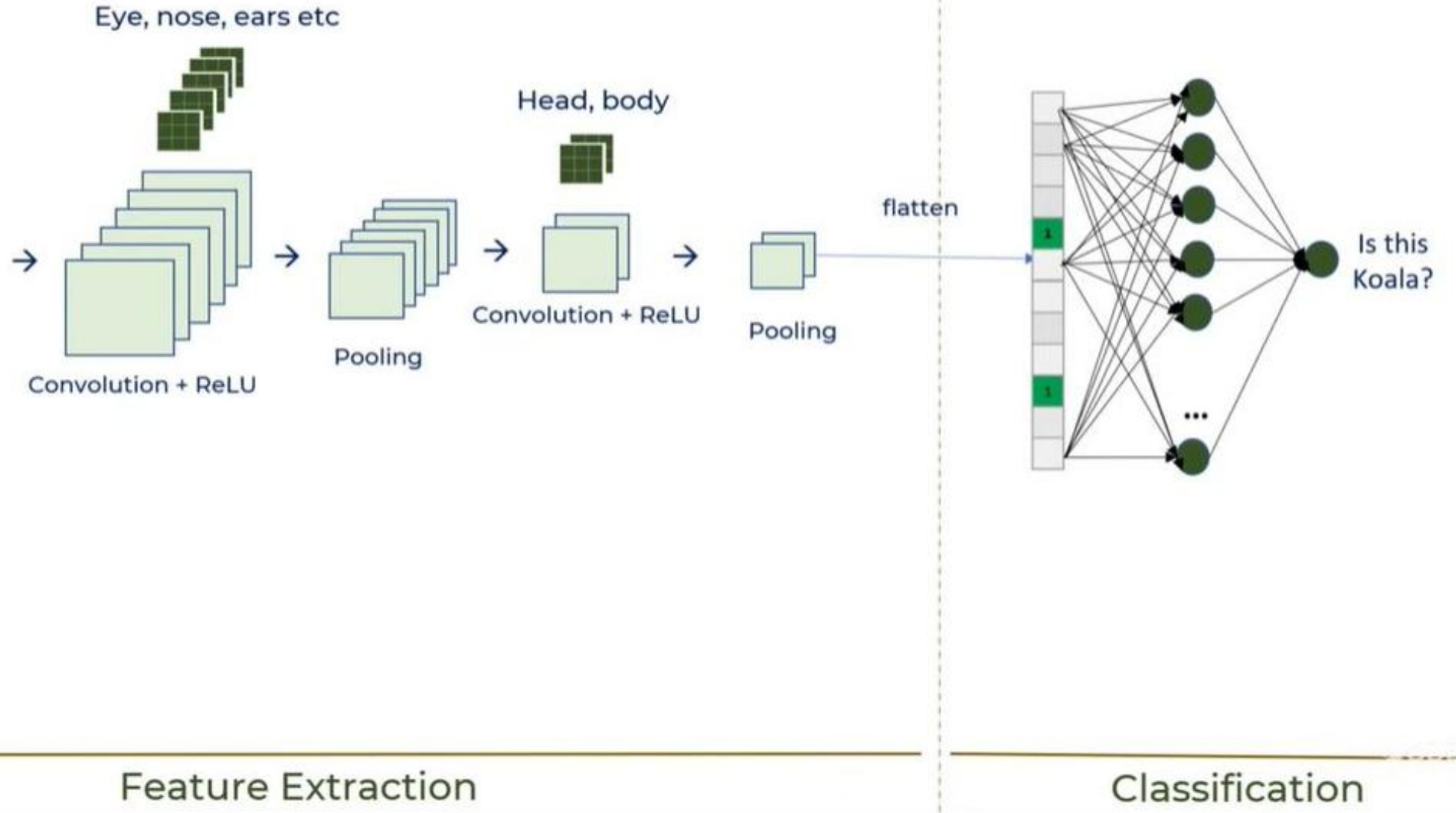
*

1	1	1
1	-1	1
1	1	1

=

	1	
		1

Loopy pattern detector



Benefits of CNN:

- Excellent for image, video, and spatial data.
- Less parameter-heavy than MLPs due to weight sharing.
- Robust to translations and distortions.

Applications:

- Image classification (e.g. ResNet, VGG)
- Object detection (YOLO, Faster R-CNN)
- Facial recognition
- Medical imaging
- Style transfer

2. Zero-Shot Learning (ZSL)

Zero-shot learning enables models to **recognize classes they've never seen during training**, by using **semantic descriptions** (e.g., attributes or word embeddings).

How it works:

- Learn from seen classes with labeled examples.
- Use semantic embeddings (e.g., word2vec, GloVe, BERT) to **bridge seen and unseen classes**.
- During inference, match features of new data to these embeddings.

If a model has never seen a “zebra” but knows about "horses" and the **attributes** of a zebra (e.g., striped, 4-legged), it can still classify it correctly.

Benefits:

- Useful when labeled data is scarce for certain classes.
- Enables generalized AI systems.

Few-Shot Learning (FSL)

Few-shot learning allows models to **learn from a very small number of examples** (e.g., 1–10 samples per class).

Method	Description
Siamese Networks	Learns to compare similarity between two inputs
Prototypical Networks	Learns a prototype (mean embedding) for each class
Meta-Learning	“Learning to learn” – adapts quickly to new tasks (e.g., MAML)
Transfer Learning	Pretrain on large data, fine-tune on small data

Benefits:

- Efficient when data collection is expensive (e.g., medical diagnosis).
- Fast adaptation to new tasks.

Relationship to CNN:

CNNs often serve as **feature extractors** in FSL and ZSL. The last layers are adapted for few/zero-shot generalization.

4. Graph Convolutional Networks (GCNs)

GCNs extend CNNs to **non-Euclidean (graph) data**, where the data is structured as nodes and edges instead of grids.

Applications:

- Social network analysis
- Recommender systems
- Molecule property prediction
- Knowledge graphs

Benefits:

- Powerful for tasks involving **relational data**.
- Captures both local and global graph structure.

How GCNs Work:

- Each node aggregates features from its **neighbors**.
- Similar to convolutions but applied over **graphs** instead of images.

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \frac{1}{c_{vu}} W^{(l)} h_u^{(l)} \right)$$

- h_v : node representation
- $\mathcal{N}(v)$: neighbors of node v
- $W^{(l)}$: learnable weight matrix
- σ : activation function (e.g., ReLU)

Concept	Purpose	Example Use-Case	Based on
CNN	Image/feature extraction	Image classification	Grid data
Zero-shot Learning	Classify unseen classes	Wildlife recognition without data	Semantic embeddings
Few-shot Learning	Learn from few samples	Medical imaging	Meta-learning, metric learning
Graph CNN (GCN)	Learn from graph data	Social network recommendation	Graph topology

Convolution

- Connections sparsity reduces overfitting
- Conv + Pooling gives location invariant feature detection
- Parameter sharing

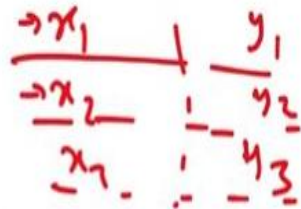
ReLU

- Introduces nonlinearity
- Speeds up training, faster to compute

Pooling

- Reduces dimensions and computation
- Reduces overfitting
- Makes the model tolerant towards small distortion and variations

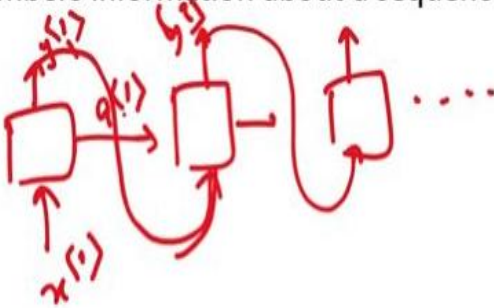
Introduction



- Output from previous step are fed as input to the current step.
- Has **Hidden State** which remembers information about a sequence.

- Have Memory.

$h_t = f(h_{t-1}, x_t)$



1. CNN (Convolutional Neural Network)

- **Purpose:** Mainly used for processing **grid-like data** such as images, videos, and spatial data.
- **Architecture:** Uses **convolutional layers** to automatically learn spatial hierarchies of features by applying filters/kernels.
- **Key Idea:** Extracts **local features** by sliding filters over input data.
- **Data Handling:** Processes data with fixed-size input (e.g., images with width × height × channels).
- **Strength:** Excellent at capturing spatial relationships and patterns (e.g., edges, textures).
- **Common Applications:** Image classification, object detection, image segmentation, video analysis.

2. RNN (Recurrent Neural Network)

- **Purpose:** Designed to handle **sequential or time-series data** by maintaining a hidden state that captures information from previous steps.
- **Architecture:** Has loops within the network to pass information from one step to the next.
- **Key Idea:** Captures **temporal dependencies** by processing data sequentially.
- **Data Handling:** Accepts sequences of variable length.
- **Strength:** Models temporal or sequential patterns (e.g., sentences, speech, time series).
- **Limitation:** Struggles with **long-term dependencies** due to vanishing/exploding gradients.
- **Common Applications:** Language modeling, speech recognition, sequence prediction.

3. LSTM (Long Short-Term Memory)

- **Purpose:** A special kind of RNN designed to overcome RNN's vanishing gradient problem, capable of learning **long-term dependencies**.
- **Architecture:** Uses **gates** (input, forget, output) to control information flow and maintain memory over long sequences.
- **Key Idea:** Can selectively **remember or forget** information using gated mechanisms.
- **Data Handling:** Like RNN, works with sequential data but better at preserving context over many time steps.
- **Strength:** Excellent for long sequences where context from far earlier in the sequence matters.
- **Common Applications:** Machine translation, speech synthesis, time-series forecasting, text generation.

What is an RNN?

A Recurrent Neural Network (RNN) is a type of neural network designed to **handle sequential data** (e.g., text, time series, audio). Unlike feedforward networks, RNNs have **loops** that allow **information to persist across time steps**.

Why use RNNs?

- To process data where **order matters**.
- To **retain memory** of previous inputs in the sequence.

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

How RNNs Work:

At each time step t , RNN takes:

- Current input x_t
- Previous hidden state h_{t-1}

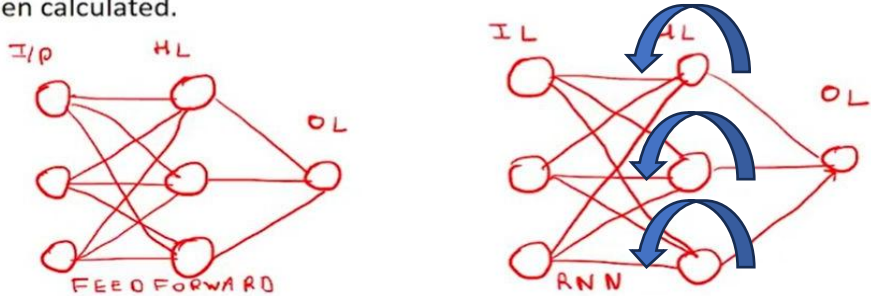
Where:

- W_{xh} : Input → Hidden weights
- W_{hh} : Hidden → Hidden (recurrent) weights
- W_{hy} : Hidden → Output weights

Recurrent Neural Network



- RNN is a feedback network in which signal can flow in both directions using close loop.
- **RNN** is a kind of ANN mainly used in speech recognition & natural language processing (NLP) to predict independent next word of sentence.
- In RNN, hidden layer has a memory to remembers information about sequence that what has been calculated.

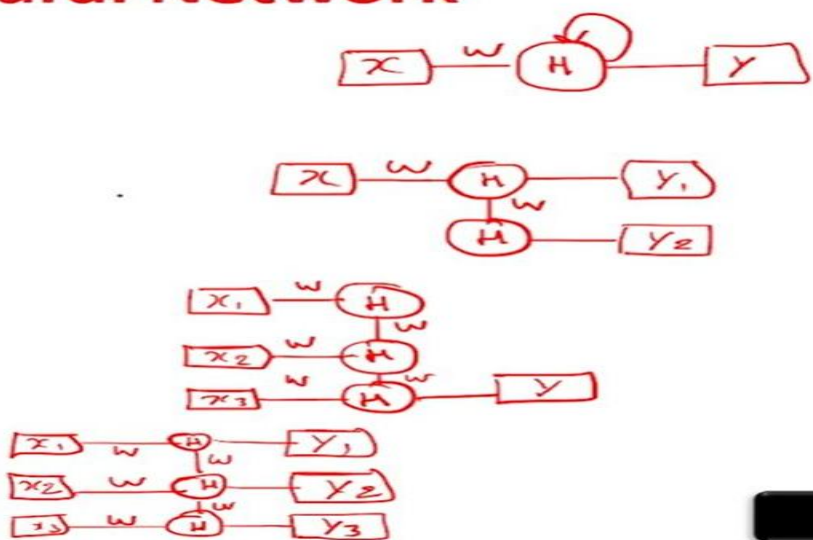


Input and out put are independent

Recurrent Neural Network

Types of RNN:

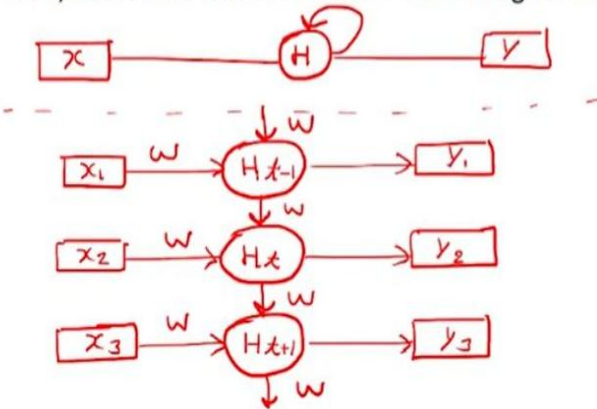
- One to one RNN:
- One to many RNN:
- Many to one RNN:
- Many to many RNN:



Recurrent Neural Network



- In RNN, output from previous step are fed as input to current step unlike feed forward network in which input and output are independent to each other.
- used to time series prediction to remember inf through time & extend pixel neighborhood.
- It use to identify patterns in data sequences, such as text, genomes, handwriting, the spoken word, numerical times series data emanating from sensors, stock markets.



Applications
Matching next letter or word
Google translation
Sentiment analysis

Issue # 1: No fixed size of neurons in a layer



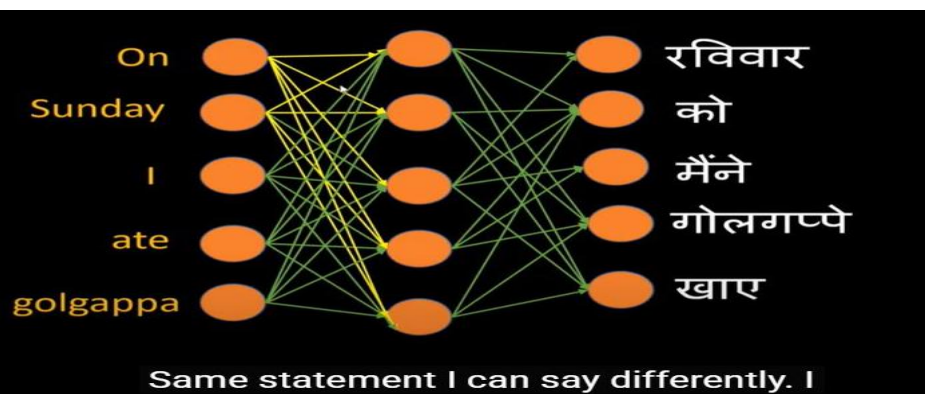
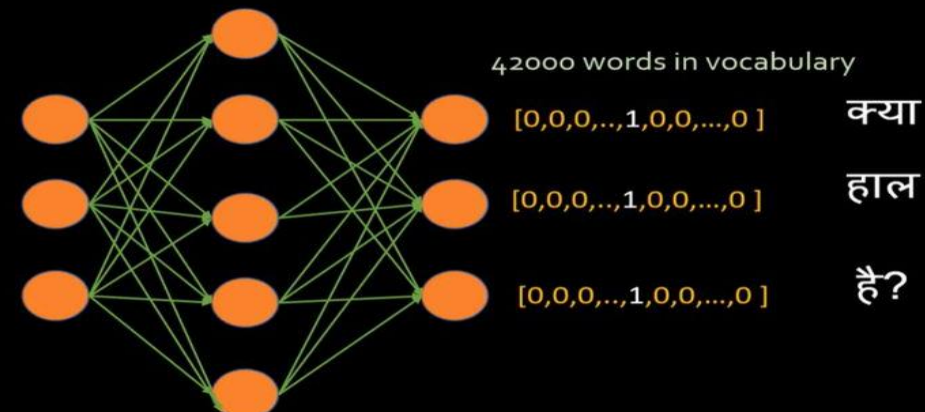
Issue # 2: Too much computation

25000 words in vocabulary

how $\rightarrow [0,0,0,\dots,1,0,0,\dots,0]$

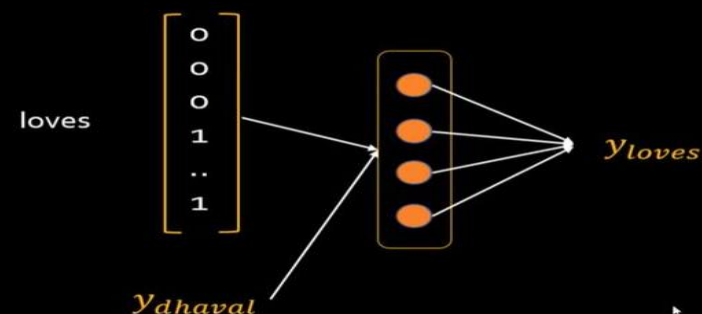
are $\rightarrow [0,1,0,0,0,\dots,0,0,\dots,0]$

you? $\rightarrow [0,0,0,0,\dots,0,0,1,0,0]$

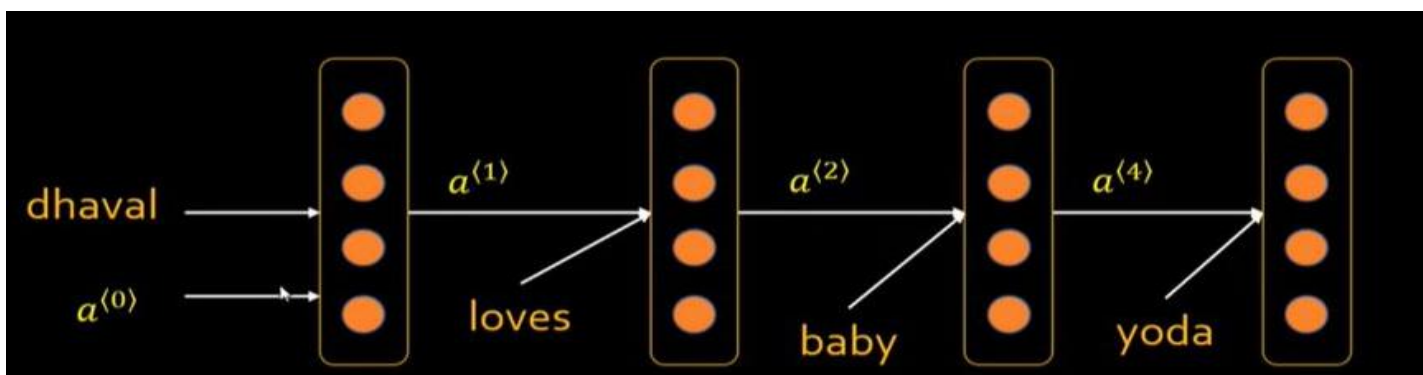
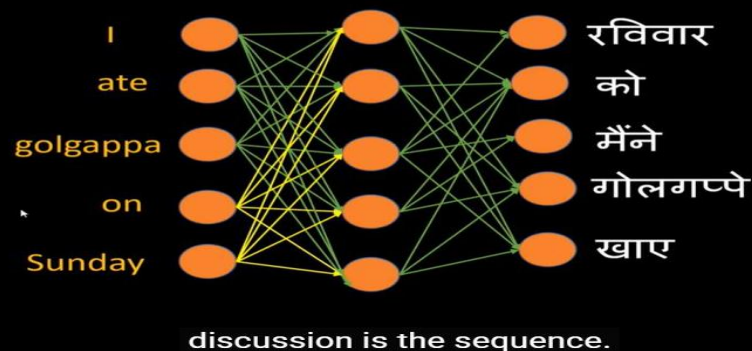


Named Entity Recognition

Dhaval loves baby yoda

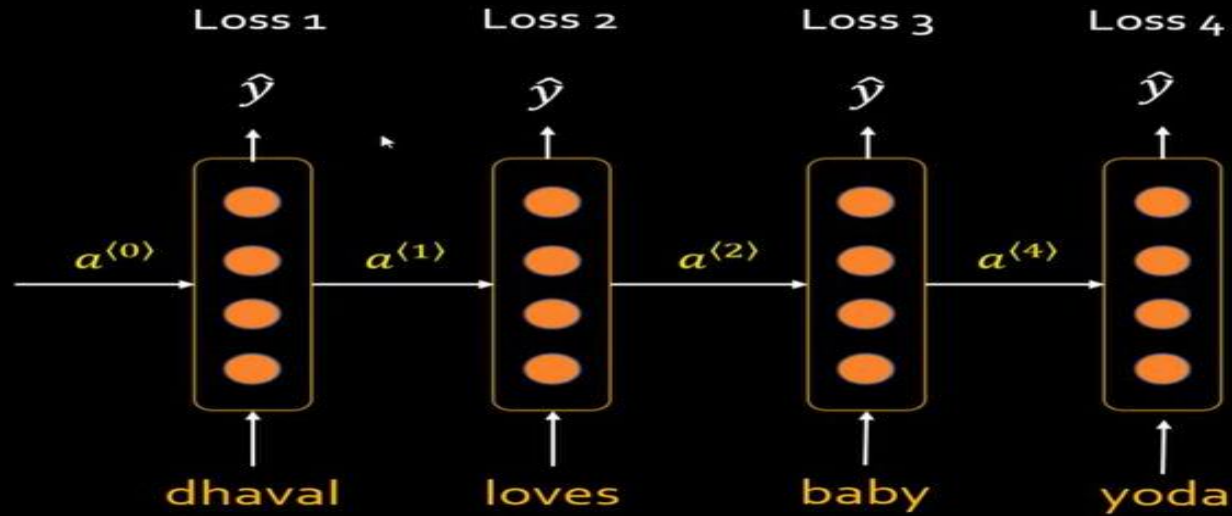


Issue # 3: Parameters are not shared



Training

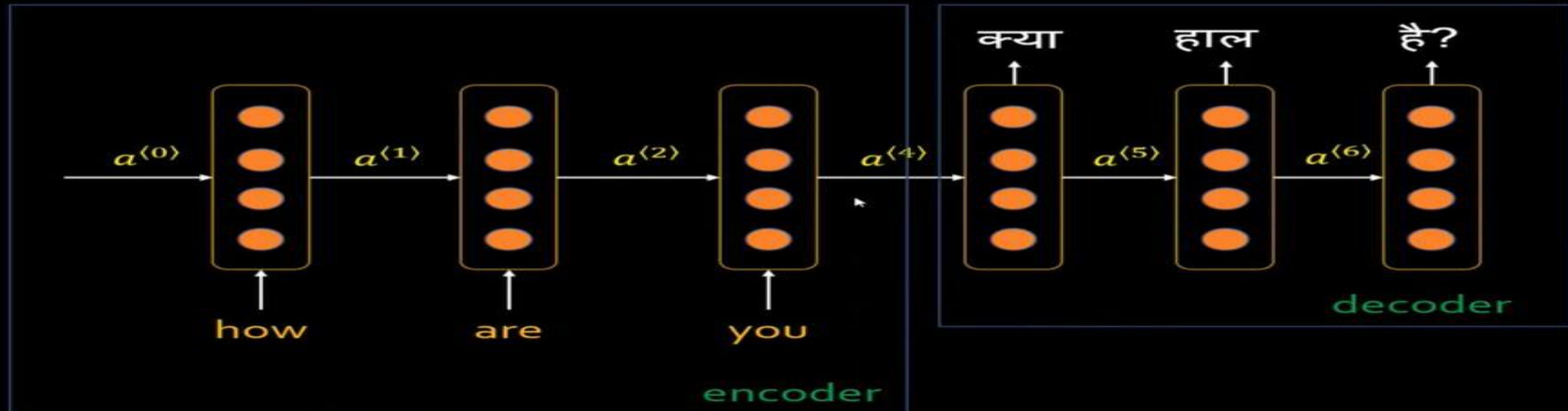
Dhaval loves baby yoda → 1 0 1 1

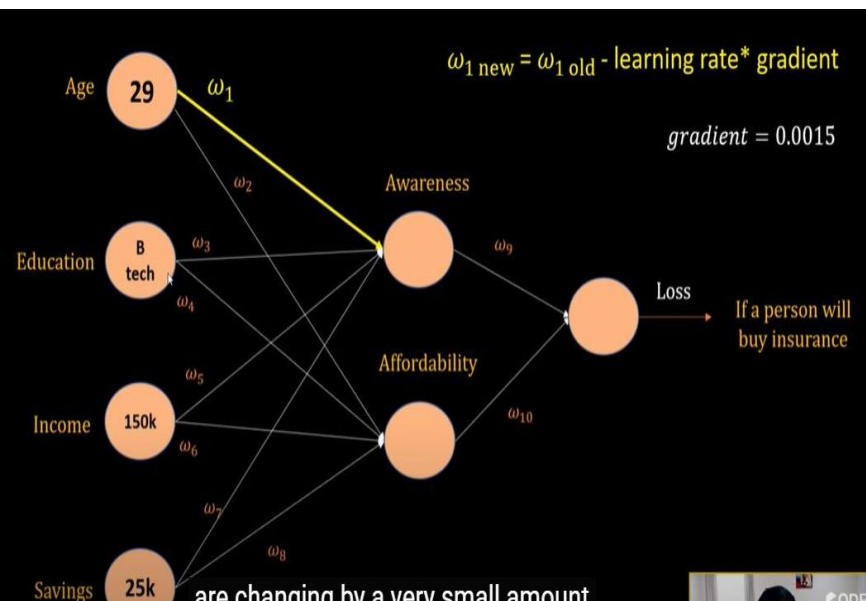
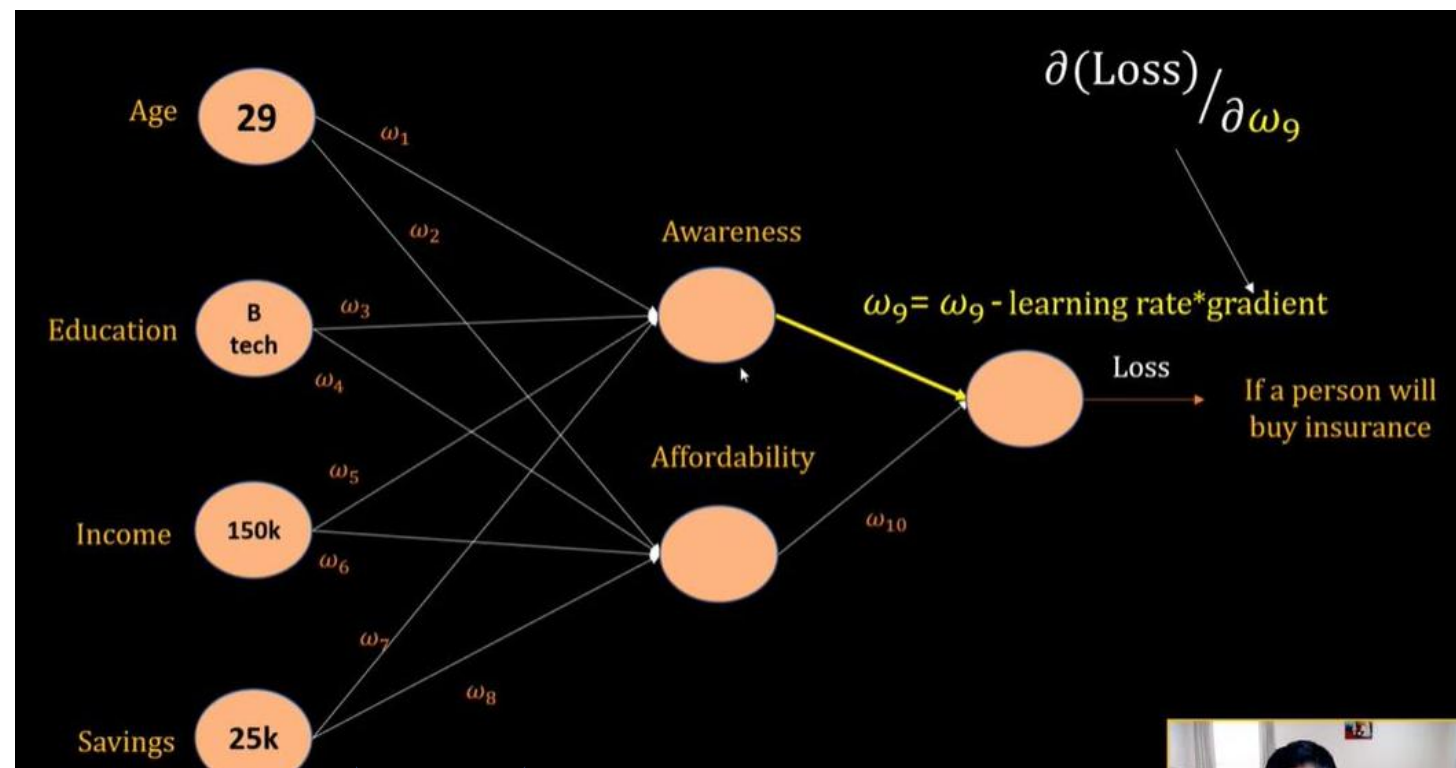
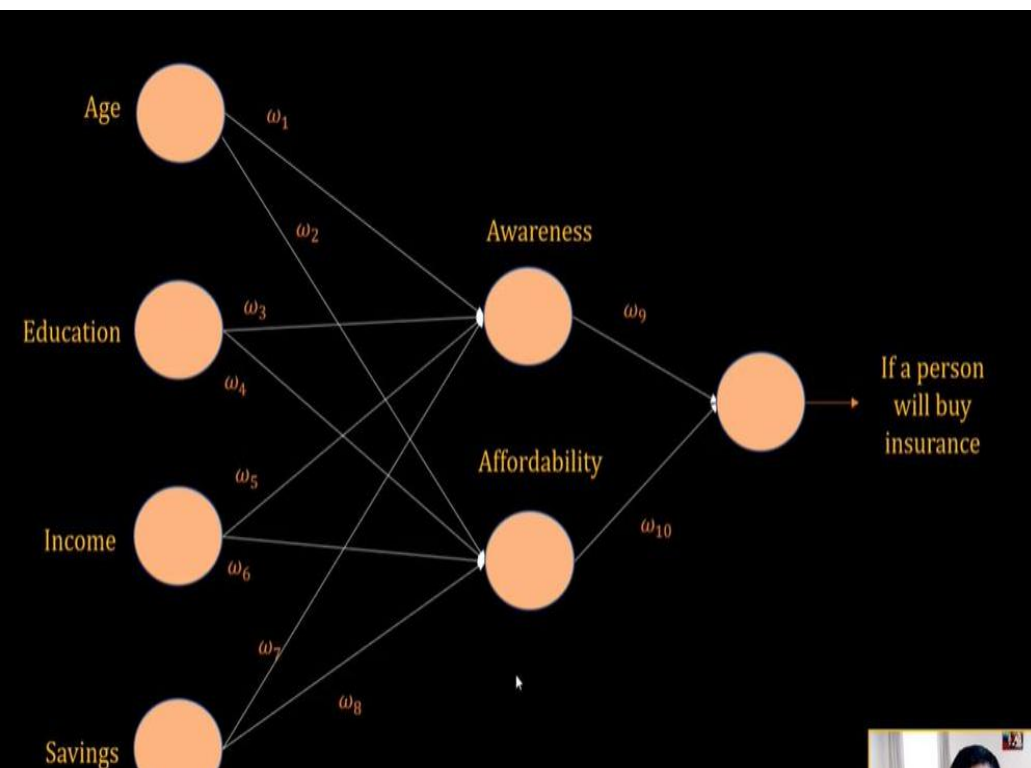


$$\text{Total Loss} = \text{Loss 1} + \text{Loss 2} + \text{Loss 3} + \text{Loss 4}$$

Language translation

how are you? क्या हाल है?





As number of hidden layers grow, gradient becomes very small and weights will hardly change. This will hamper the learning process.

Vanishing Gradients

What is "Big Data"?

Big data refers to **very complex datasets** that for traditional software process efficiently.

It's defined by the 3 Vs (sometimes 5 Vs):

1. **Volume** – Massive amount of data (e.g., petabytes)

2. **Velocity** – Data is generated rapidly and continuously from real-time social media

- **Natural Language Processing (NLP):**
- Sentiment analysis
- Machine translation
- Text generation
- **Speech recognition**
- **Time series forecasting**
- **Music generation**

Variant	Description	Best for
Vanilla RNN	Basic RNN with short-term memory	Simple sequences
LSTM (Long Short-Term Memory)	Handles long-term dependencies via gates	Long sequences
GRU (Gated Recurrent Unit)	Simplified version of LSTM, fewer parameters	Faster training
Bidirectional RNN	Processes sequence in both directions	NLP tasks
Encoder-Decoder (Seq2Seq)	Used in translation, summarization	Input/output sequences

Challenges of RNNs:
Solutions: Use **LSTM**, **GRU**, **attention mechanisms**, or switch to **Transformers**.

Problem	Explanation
Vanishing gradient	Hard to learn long-term dependencies
Exploding gradient	Large updates destabilize learning
Slow training	Cannot parallelize across time steps easily
Memory bottleneck	Hidden states may not capture all relevant info

- Benefits of RNNs:**
- Handles variable-length sequences
 - Captures temporal dynamics
 - Shares parameters across time

- Limitations:**
- Hard to train on long sequences
 - Sequential nature limits parallelization
 - Replaced in many applications by **Transformers**

```
import torch
import torch.nn as nn
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=1)
input_seq = torch.randn(5, 3, 10) # (seq_len, batch, input_size)
hidden = torch.zeros(1, 3, 20)    # (num_layers, batch, hidden_size)
output, hidden = rnn(input_seq, hidden)
print(output.shape) # torch.Size([5, 3, 20])
```

What is Attention?

Attention mechanisms in neural networks allow models **to focus on the most relevant parts of the input when generating outputs**—mimicking how humans pay selective attention.

Why is Attention Important?

Solves the **bottleneck** problem in sequence models (e.g., RNNs, LSTMs), which try to compress all input into a single vector.

Key idea: Instead of processing all input equally, attention **weights** input elements **differently**, depending on their importance to the task.

- Enables **better long-range dependency handling**.
- Forms the **foundation of Transformers**, which have replaced RNNs in many NLP tasks.

1. Basic Attention Mechanism

Core components:

- **Query (Q):** What we're looking for.
- **Key (K):** What each input element offers.
- **Value (V):** What we actually use from the input.

Scoring and Weighting:

- The softmax gives **attention weights**.
- These weights are used to compute a **weighted sum of values (V)**.

$$\text{Score}(Q, K) = Q \cdot K^T$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Type	Description
Additive Attention	Computes score using a feedforward network (used in early models like Bahdanau attention)
Dot-product Attention	Simpler, faster; uses dot products between Q and K
Self-Attention	Query, Key, and Value come from the same input sequence
Cross-Attention	Q comes from one source, K and V from another (used in encoder-decoder models)
Multi-Head Attention	Runs multiple self-attention mechanisms in parallel and concatenates outputs

3. Self-Attention (used in Transformers)

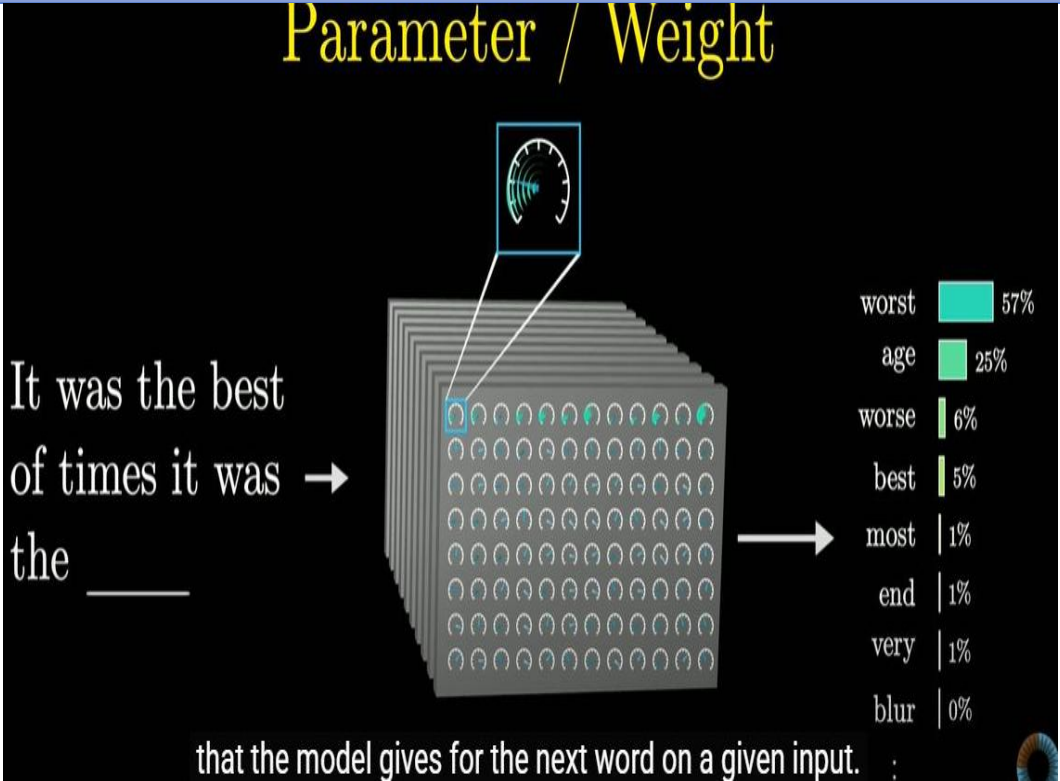
Self-attention relates **each position in a sequence to every other position**, allowing the model to understand **context and relationships**.

Let’s say we input a sentence:

arduino

"The cat sat on the mat"

- Self-attention lets the model understand that "cat" is the subject and "sat" is the verb, even if they are several words apart.



4. Multi-Head Attention

Instead of one attention computation, the model computes multiple "heads":

$$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,\dots,\text{head}_h)W_O$$

Each head learns different relationships in the data.

Used in:

- Transformer architecture (BERT, GPT, T5)
- Vision Transformers (ViT)
- Machine Translation
- Speech & time-series modeling

Benefit	Why It Matters
Focus on relevant input	Enhances model interpretability & performance
Handles long sequences	Better than RNNs/LSTMs for long-range dependencies
Parallelizable	Especially in Transformers, unlike RNNs
General-purpose	Works in NLP, vision, audio, graphs, etc.

Task	How Attention Helps
Machine Translation	Focus on relevant source words while generating each target word
Text Summarization	Extract most important parts of the text
Image Captioning	Attend to parts of the image while generating words
Speech Recognition	Align speech frames to text output

- A **Python example** of attention in PyTorch or TensorFlow?
- A **visual diagram** of self-attention or multi-head attention?
- A comparison of **Attention vs RNN/LSTM**?

Transformers are a type of neural network architecture that have revolutionized the field of deep learning, particularly in **natural language processing (NLP)**, but also in areas like **computer vision, time-series analysis, and more.**

What Is a Transformer? Word presents in sequential matters

The **Transformer** architecture was introduced in the 2017 paper "**Attention is All You Need**" by **Vaswani et al.** It discards traditional recurrent structures (like RNNs or LSTMs) and instead relies entirely on **attention mechanisms** to model relationships between **elements in a sequence.**

Key Components of a Transformer

1. Input Embedding + Positional Encoding

- Converts tokens (like words) into vectors.
- Since transformers don't have recurrence or convolution, they add **positional encoding** to capture the order.

2. Multi-Head Self-Attention

- Allows the model to **weigh the importance** of other tokens in the sequence when encoding a particular token.
- Multiple attention heads allow the model to focus on different relationships.

3. Feed-Forward Neural Network (FFN)

- A position-wise fully connected network applied to each position's output from the attention layer.

4. Residual Connections + Layer Normalization

- Helps in training deep networks by preventing vanishing gradients.

5. Encoder-Decoder Structure (for sequence-to-sequence tasks)

- **Encoder:** Processes the input sequence.
- **Decoder:** Generates the output sequence, using encoder outputs and previously generated outputs.

Transformer Variants

- BERT (Bidirectional Encoder Representations from Transformers)**: Uses only the encoder part, ideal for classification, Q&A, etc.
- GPT (Generative Pre-trained Transformer)**: Uses only the decoder part (unidirectional), ideal for text generation.
- T5, XLNet, RoBERTa, LLaMA, etc.**: Variants with different improvements and training schemes.

Applications

- Machine translation
- Text summarization
- Sentiment analysis
- Text generation (e.g., ChatGPT)
- Image classification (Vision Transformers)
- Time-series forecasting

Model Name	Organization	Use Case
GPT-4 / GPT-3.5	OpenAI	General-purpose text generation
BERT (not generative)	Google	Used for understanding, not generating text
T5	Google	Text-to-text generation tasks
LLaMA	Meta	Open-source generative model
Claude	Anthropic	Conversational AI

Why Are Transformers Powerful?

- Parallelizable**: Unlike RNNs, transformers process sequences all at once (not token by token).
- Scalable**: Can be trained on massive datasets with GPUs/TPUs.
- Generalizable**: Foundation for large pre-trained models (e.g., GPT, BERT) that can be fine-tuned for many tasks.

Aspect	Tokenization	Embeddings
Purpose	Splits text into smaller units (tokens)	Converts tokens into numerical vectors
Process	Converts text like "Hello, world!" into tokens ["Hello", ",", "world", "!"] Maps tokens to high-dimensional vectors (e.g., Word2Vec, BERT)	
Role	Prepares text for further processing	Provides dense representations capturing semantic meaning
Granularity	Works at word, subword, or character level Works at vector representation level	
Example	Output ["Hello", ",", "world", "!"]	Vectors like [0.5, 0.1, ..., 0.3]

King

authority: 1

has tail?: 0

rich: 1

gender: -1

1
0
1
-1

Queen

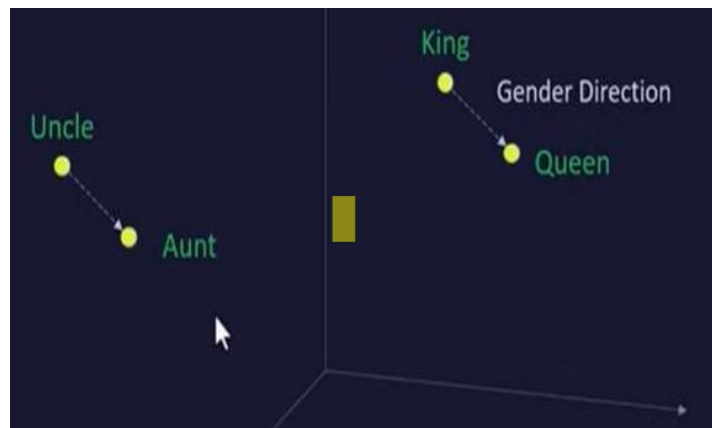
authority: 1

has tail?: 0

rich: 1

gender: -1

1
0
1
1



Limitations of Word Embeddings Models (Word2Vec)

The train will run on the track.

My package is late, help me track it.

track

0.56
-1.1
-0.32
1.21
...
-1.88



I made a rice dish called

risotto

idli

mexican rice

I made an Indian rice dish called

idli

biryani

kheer

I made a sweet Indian rice dish called

kheer

pongal

I made a **sweet** Indian rice dish called **kheer**

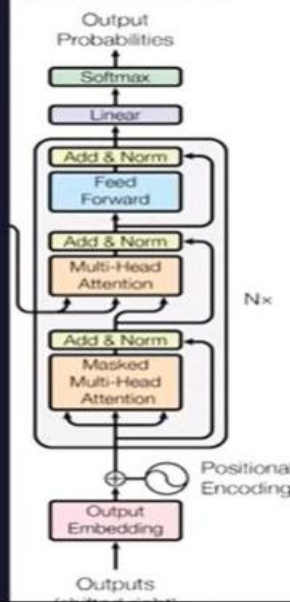
dish	rice ness	indian ness	sweet ness	dish
1.09	-0.43	0.29	0.39	1.34
-0.72	0.91	0.67	-0.11	0.75
0.66	0.18	0.52	-0.98	0.38
...	...	...	...	...
-0.8	-0.75	0.85	-0.46	-1.16

I made a sweet Indian rice dish called

-1.08	1.34	-0.98
-1.75	0.75	1.78
2.38	0.38	0.22
...	...	...
1.65	-1.16	1.29

Contextual Embedding

Decoder

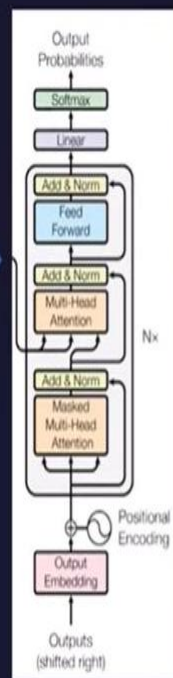


alpha	0.9 %
biryani	2.3 %
kheer	5.9 %
pongali	41.9 %
table	0.02 %
...	...
zebra	0.07 %

I made kheer

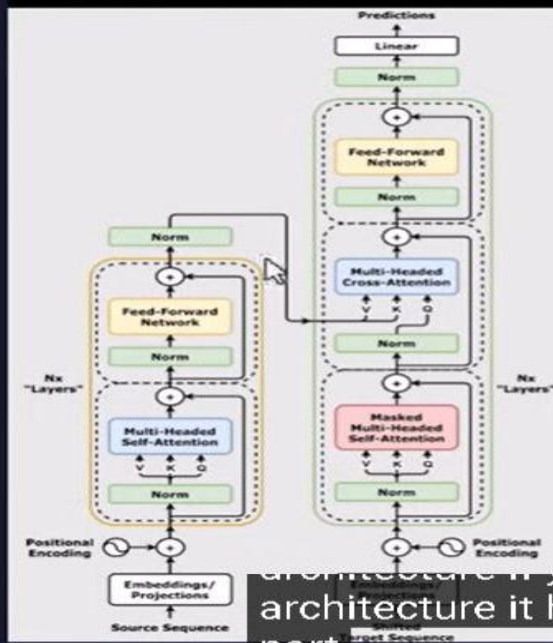
-1.08	1.34	-0.98
-1.75	0.75	1.78
2.38	0.38	0.22
...	...	...
1.65	-1.16	1.29

Contextual Embedding

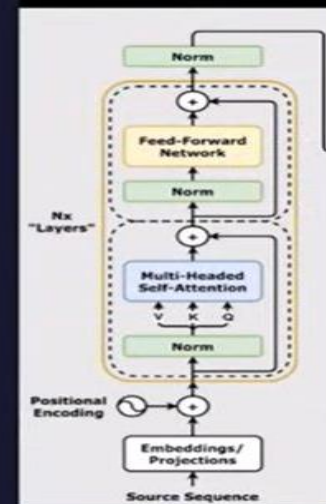


आम	1.1 %
बनाई	0.9 %
खीर	12.0 %
सूरज	1.9 %
...	...
<END>	57.8 %

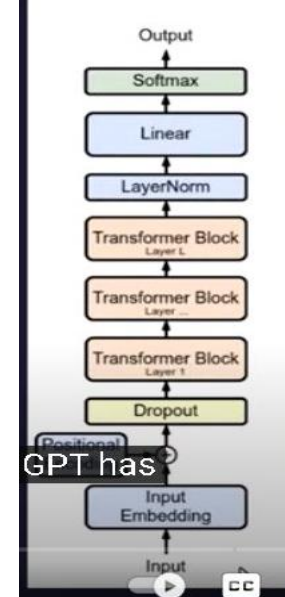
Generic Architecture



BERT



GPT



architecture if you look at the architecture it has only the encoder part

Graph Attention Networks (GATs) are a type of neural network specifically designed to operate on **graph-structured data**. They **extend the concept of attention (as used in Transformers) to graphs**, **where data is not sequential (like in text) but instead made up of nodes and edges**.

Why Graphs?

In many real-world problems, data is naturally represented as a **graph**:

- Social networks (users are nodes, friendships are edges)
- Citation networks (papers are nodes, citations are edges)
- Molecules (atoms as nodes, chemical bonds as edges)
- Recommendation systems, traffic networks, etc.

Traditional neural networks struggle with these irregular, non-Euclidean structures. That's where **Graph Neural Networks (GNNs)** — and especially **GATs** — come in.

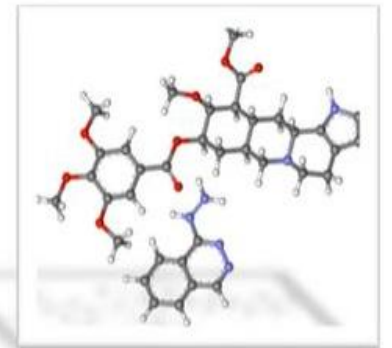
What Is a Graph Attention Network (GAT)?

Introduced in the 2017 paper:

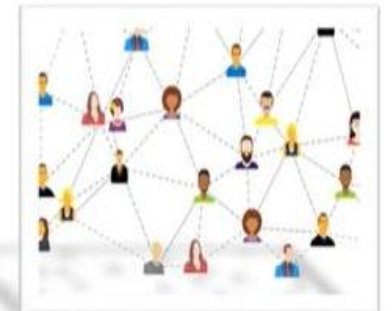
"Graph Attention Networks" by Veličković et al.

GATs are an evolution of Graph Convolutional Networks (GCNs), **but instead of treating all neighbors equally or using fixed weights, they use attention** to learn how important each neighbor is when updating a node's representation.

Graph Data is everywhere



Medicine / Pharmacy



Social Networks

How GAT Works (High-Level Steps)

1. **Node Features:** Each node i starts with some feature vector h_i .

2. Attention Coefficient Calculation:

For every node i and its neighbor j , compute an attention score α_{ij} that indicates **how important** node j 's information is to node i .

This is typically done with a small neural network (like a feedforward layer) using:

Graph Neural Networks



$$e_{ij} = \text{LeakyReLU}(a^T [Wh_i \parallel Wh_j])$$

Then apply softmax over neighbors of i :

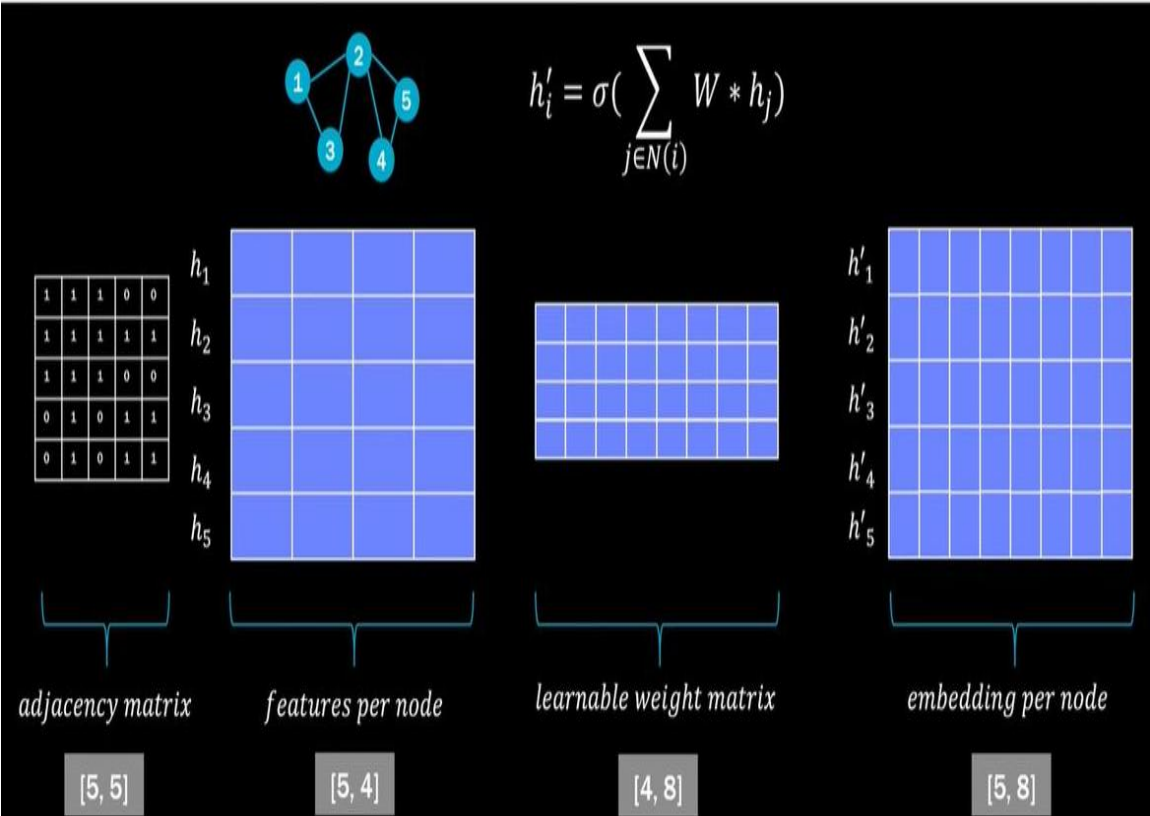
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

3. Feature Aggregation:

The new representation for node i is a weighted sum of its neighbors' transformed features:

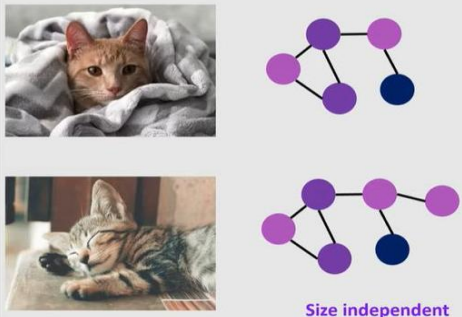
$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \right)$$

” Ask ChatGPT



Problem: Graph Data is different!

Difference 1: Size and Shape



Size independent

Difference 2: Isomorphism



Permutation invariance

Difference 3: Grid structure



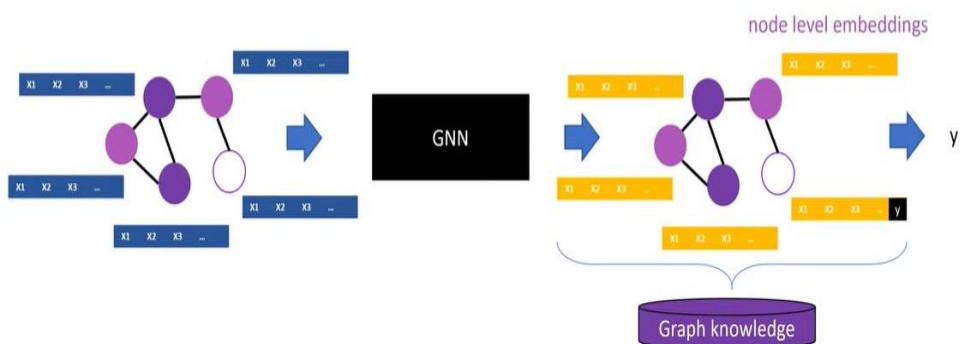
Non-euclidean space

Adjacency matrix as input

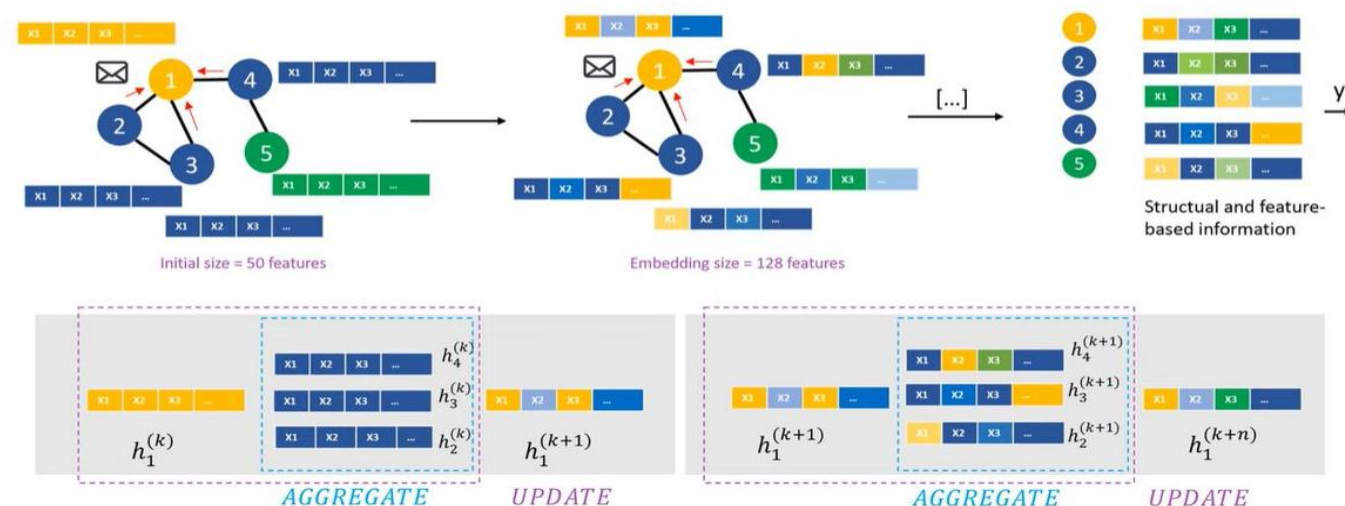
Fundamental Idea of GNNs

>> Learning a for neural networks suitable representation of graph data. <<

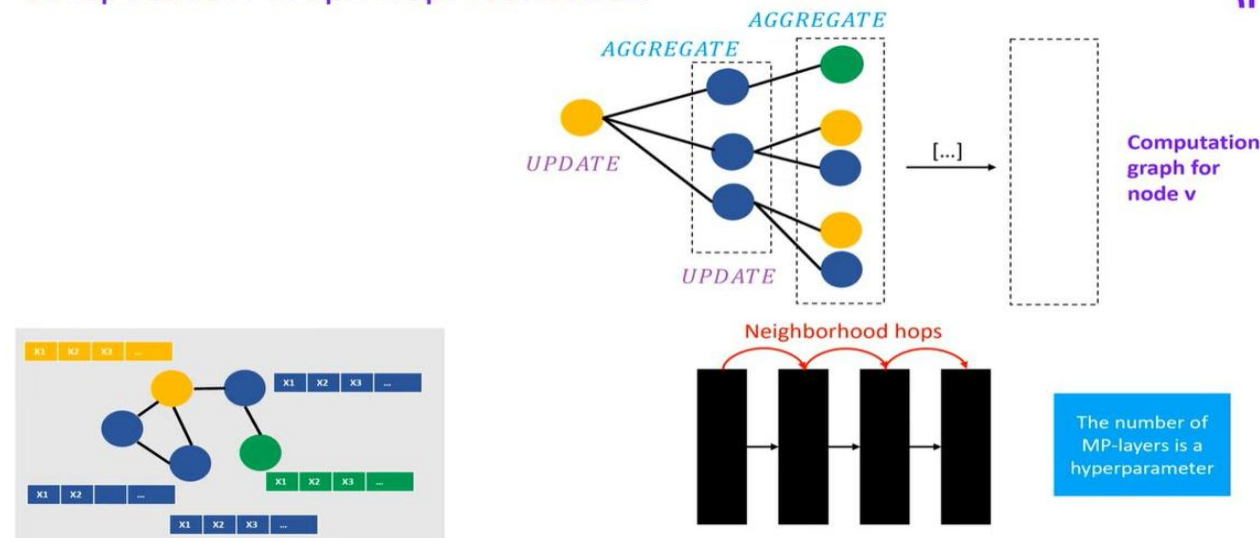
= Representation learning



What is happening in the Message Passing Layers?



Computation Graph Representation





Graph Convolutional Networks,
Kipf and Welling [2016]

$$h_v^{(k)} = \sigma \left(\text{Self-loop} \right)$$

$v \in \mathcal{V}$

Multi-Layer-Perceptron as
Aggregator, Zaheer et al. [2017]

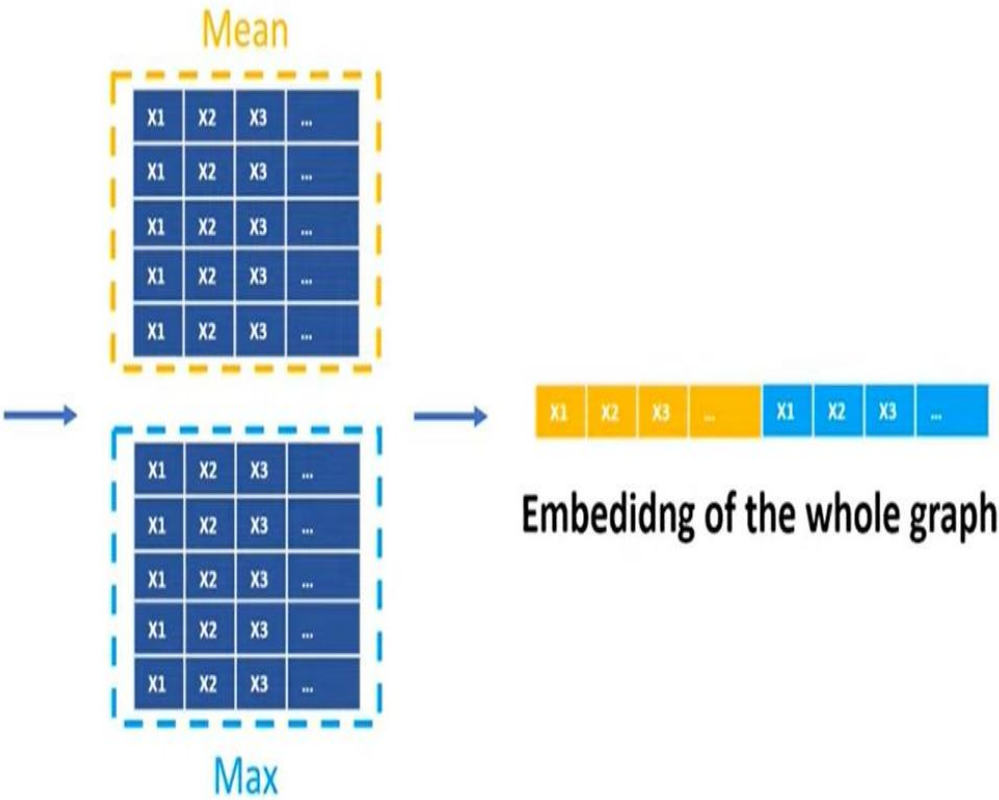
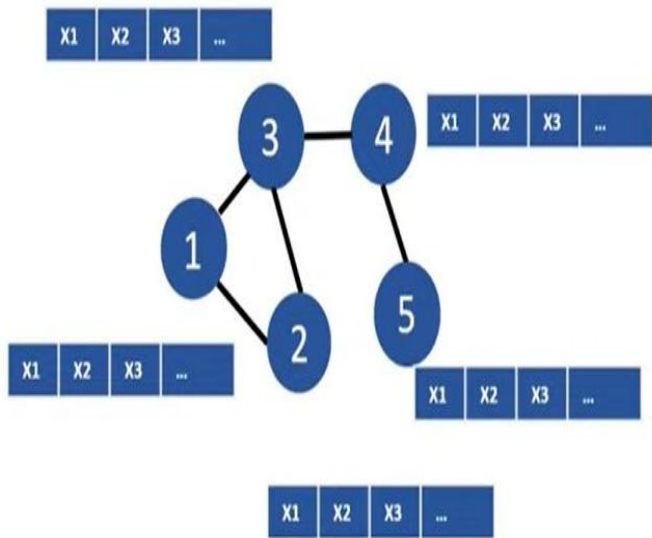
Aggregated message

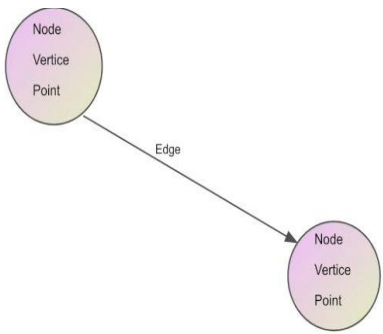
$$m_{\mathcal{N}(u)} = \text{MLP}_{\theta} \left(\text{trainable!} \right)$$

Graph Attention Networks,
Veličković et al. [2017]

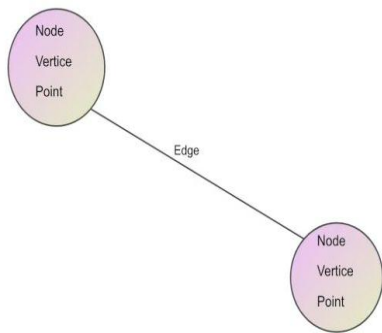
$$m_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha$$

Gated Graph Neural Networks,
Li et al. [2015]

$$h_u^{(k)} = \text{GRU}(h_u^{(k-1)})$$


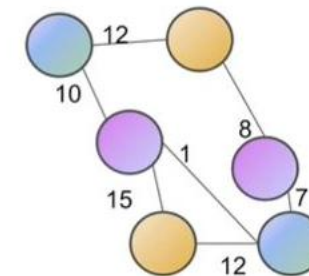
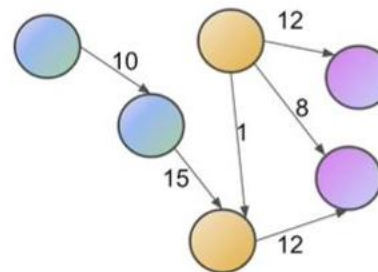


Directed

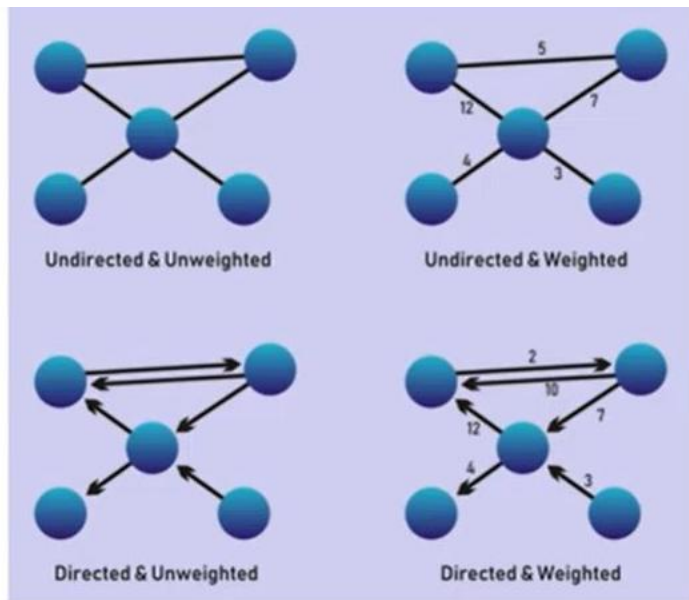


Undirected

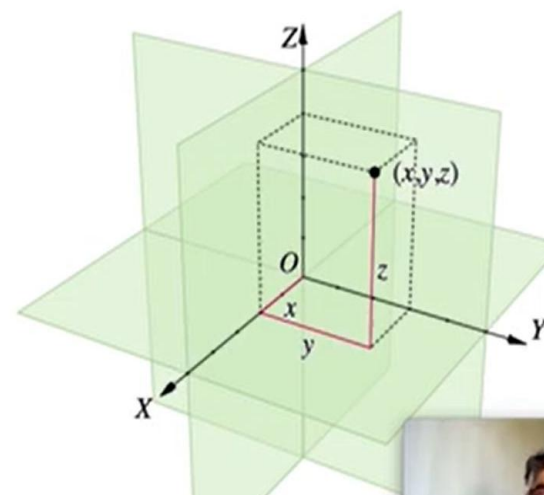
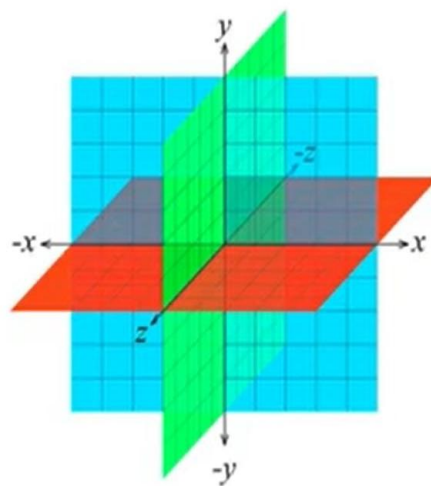
Weighted Graphs



- A Weighted graph is a graph with edges labeled by the numbers
 - I.e. Distance, quantity, price, value etc.
- A weight is a numerical value attached to each individual edge.



Euclidean Space - 3 Dimensional (x,y,z) Plane



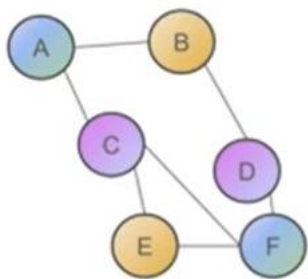
Why we should use Graphs?

- Give intuitive representation to abstract concepts i.e. relationships and information.
- Intuitively visual representation of information.
- Form a Natural basis for analyzing relationships in a Social context.
- Break down complex problems into simpler representations
- Transform the complex problems into representations from different perspectives.

Traditional Graph Analysis Methods

- Searching algorithms, e.g. BFS, DFS
- Shortest path algorithms, e.g. Dijkstra's algorithm, Nearest Neighbour
- Spanning-tree algorithms, e.g. Prim's algorithm
- Clustering methods, e.g. Highly Connected Components, k-mean

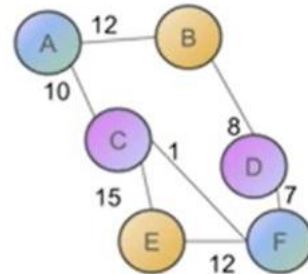
Mathematical Representation of Graph



Set of Vertices
- $V = \{A, B, C, D, E, F\} \rightarrow$

Set of Edges
- $E = \{AB, AC, BD, CE, CF, DF, EF\}$
- $E = \{(A,B), (A,C), (B,D), (C,E), (C,F), (D,F), (E,F)\}$

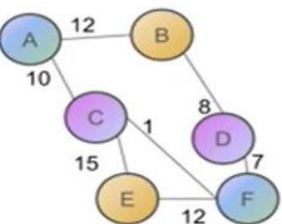
$Graph\ G = (V, E)$



Set of Vertices
- $V = \{A, B, C, D, E, F\} \rightarrow$

Set of Edges
- $E = \{(A,B,12), (A,C,10), (B,D,8), (C,D,1), (C,E,15), (C,F,12), (D,F,7), (E,F,12)\}$

Adjacency Matrix

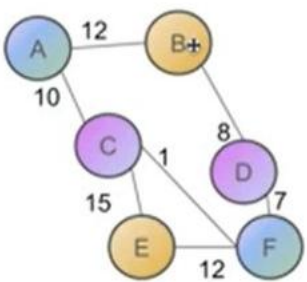


- Adjacency matrix a 2D square matrix
- Each node in the graph has an entry in both dimensions.
- Unweighted graph as T/F or 1/0 values
- Weighted graph as weights, no weights means -1

Representation:

- $A = N \times N$

Neighbors



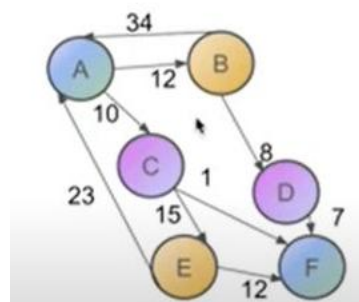
Neighbors:

- Two nodes that are connected with an edge are called neighbors.

Given Example:

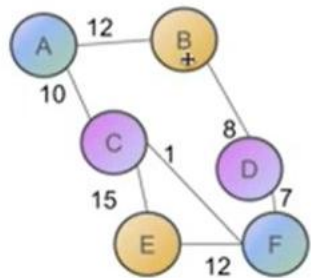
- B and C are neighbors for A
- E and F are neighbors for C
- D is neighbor for B
- F is neighbor for C, D and E

Edge List - Directed



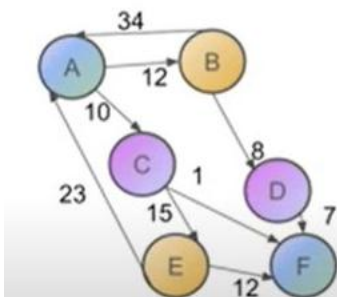
A	B	12
A	C	10
B	D	8
C	E	15
C	F	1
D	F	7
E	F	12
E	A	23
B	A	34

Adjacency Matrix - Weighted & Undirected



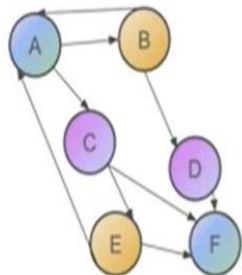
	A	B	C	D	E	F
A	-1	12	10	-1	-1	-1
B	-1	-1	-1	8	-1	-1
C	-1	-1	-1	-1	15	1
D	-1	-1	-1	-1	-1	7
E	-1	-1	-1	-1	-1	12
F	-1	-1	-1	-1	-1	-1

Adjacency Matrix - Weighted & Directed



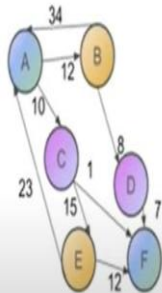
	A	B	C	D	E	F
A	-1	12	10	-1	-1	-1
B	34	-1	-1	8	-1	-1
C	-1	-1	-1	-1	15	1
D	-1	-1	-1	-1	-1	7
E	23	-1	-1	-1	-1	12
F	-1	-1	-1	-1	-1	-1

Adjacency Matrix - Un-Weighted & Directed



	A	B	C	D	E	F
A	F	T	T	F	F	F
B	T	F	F	T	F	F
C	F	F	F	F	T	T
D	F	F	F	F	F	T
E	T	F	F	F	F	T
F	F	F	F	F	F	F

Adjacency Matrix - Weighted & Directed



	A	B	C	D	E	F
A	-1	12	10	-1	-1	-1
B	34	-1	-1	8	-1	-1
C	-1	-1	-1	-1	15	1
D	-1	-1	-1	-1	-1	7
E	23	-1	-1	-1	-1	12
F	-1	-1	-1	-1	-1	-1

A_{AB}	A-B
A_{AC}	A-C
A_{AF}	A-C-E-F
A_{CD}	C-E-A-B-D
A_{BC}	X

Complete Graph

- All elements of adjacency matrix **A** are 1/T, except along the diagonal
- Path exists for each and every node

Sparse Graph

- Not all elements of adjacency matrix **A** are 1/T, lots of values are -1 or F/o
- Not every node is connected to each other

Node Attribute Matrix / Feature Matrix (X)



- The features or attributes of each node
- A Graph with N nodes with the size of node attributes as F ,
 - Matrix Shape = $N \times F$

Example:

Document 1: I travelled to Himalaya.

Document 2: I travelled to Las Vegas Nevada

Corpus: {I, travelled, to, Himalaya, Las, Vegas, Nevada}

Size of Corpus (F) = 7

	Document 1	Document 2
I	1	1
Travelled	1	1
to	1	1
Himalaya	1	0
Las	0	1
Vegas	0	1
Nevada	0	1

The shape of node attributes matrix $X = N \times F = 2 \times 7 = 14$

1. Multi-Head Attention (like Transformers):

Multiple attention heads are used to stabilize and enrich learning:

- Concatenate or average the outputs from different heads.

Key Benefits of GATs

- **No need for knowing the whole graph structure ahead of time**
- **Adaptive weighting** of neighbor nodes using attention
- **Handles graphs with nodes having varying degrees (number of neighbors)**

Use Cases

- Node classification (e.g., classifying users in a social graph)
- Link prediction (e.g., predicting future friendships or chemical bonds)
- Graph classification (e.g., predicting molecular properties)
- Traffic flow prediction, knowledge graph completion

Feature	GCN	GAT
Neighbor weighting	Fixed (based on degree)	Learned (via attention)
Expressivity	Moderate	Higher (adaptive learning)
Compute cost	Lower	Slightly higher
Performance	Good	Often better on complex graphs

Transfer Learning is a powerful technique in **neural and deep learning** where a model trained on one task is reused (wholly or in part) for a different but related task.

What Is Transfer Learning?

Transfer learning leverages **knowledge from a pre-trained model** to solve a new problem, often when you have **limited labeled data** for the new task.

For example:

- A model trained on **ImageNet** to classify 1000 objects can be reused to classify **medical images** or **wildlife species**.

How It Works

Here's a high-level view of how transfer learning is used in practice:

1. Pretraining Phase

- A deep network (e.g., CNN, Transformer) is trained on a **large dataset** (e.g., ImageNet for images, Wikipedia for NLP).
- The model learns general-purpose features like edges, shapes, sentence structure, or semantic relationships.

2. Transfer Phase (Fine-Tuning or Feature Extraction)

- You take the pre-trained model and:
 - **Freeze** some of the earlier layers (so they don't change).
 - **Fine-tune** the later layers using your **task-specific dataset**.
 - Or add new output layers suited to your task.

Common Transfer Learning Strategies

Strategy	Description
Feature Extraction	Use the pre-trained model as a fixed feature extractor (freeze all layers except the classifier).
Fine-Tuning	Unfreeze some or all layers of the pre-trained model and retrain them on new data
Partial Transfer	Transfer only a subset of weights/layers from the source model.

Popular Models for Transfer Learning

For Images:

- ResNet, VGG, Inception, EfficientNet (trained on ImageNet)
- Use in medical imaging, face recognition, etc.

For NLP:

- BERT, GPT, RoBERTa, T5
- Pretrained on massive corpora → fine-tuned for sentiment analysis, question answering, summarization, etc.

Why Transfer Learning Works

Deep neural networks tend to learn **general features** in the early layers and **task-specific features** in the later ones. By reusing the general features, transfer learning enables:

- Faster training
- Better accuracy with less data
- Reduced need for large computational resources

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, models

# Load pre-trained VGG16 without top classification layer
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze base model
base_model.trainable = False
a
# Add custom classification head
model = models.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax') # for 10-class classification
])
```

```
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)
```

----- 4. Define a simple neural network -----

```
class BMIR regressor(nn.Module):
    def __init__(self):
        super(BMIR regressor, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(3, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 1)
        )

    def forward(self, x):
        return self.model(x)
```

```
model = BMIR regressor()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

----- 5. Train the model -----

```
num_epochs = 200
```

```
losses = []
```